

AI-Based MRI Image Classification for Brain Tumor
Categorization

SUSAN ARYAL

(LCID: LC00017002100)

FINAL YEAR PROJECT REPORT SUBMITTED IN PARTIAL
FULFILMENT FOR THE DEGREE OF BACHELOR
(HONOURS) OF INFORMATION TECHNOLOGY

FACULTY OF COMPUTER SCIENCE IN MULTIMEDIA
LINCOLN UNIVERSITY COLLEGE
MALAYSIA

DATE: 2/20/2026

DECLARATION OF AUTHENTICITY

I, Susan Aryal, a student of BIT at Texas College of Management and IT, hereby declare that the project report entitled "Brain Tumor Detection and Classification Using Machine Learning," submitted by me for the award of the degree of Bachelor of Information Technology (Honours), is my own original work. I further declare that the matter embodied in this project report has not been submitted by me for the award of any other degree or diploma of any other institute, and I have followed the ethical guidelines for research and writing throughout the process. Furthermore, all sources of information, datasets, and literature used in this project, including the Masoud Nickparvar Brain Tumor MRI Dataset and Google Machine Learning resources, have been duly acknowledged and cited in the reference section. Finally, I confirm that the experimental results, including the reported accuracy of 86%, are based on actual tests performed using the developed model.

Susan Aryal

LC ID: LC00017002100

Date: 2/20/2026

ACKNOWLEDGEMENTS

I would like to express my deepest gratitude to all those who have helped me with the successful completion of this project.

First and foremost, I am extremely grateful to my project supervisor, Er. Saroj Ghimire, for their invaluable guidance, constant encouragement, and technical insights throughout the development of this brain tumor classification system. Their expertise in Machine Learning and Computer Vision was instrumental in achieving an accuracy of 86%.

I also extend my sincere thanks to the Texas Collage of Management and IT for providing the necessary infrastructure and resources to conduct this research.

A special word of thanks to Masoud Nickparvar for making the Brain Tumor MRI Dataset available on Kaggle, which served as the foundation for training and testing my model. I also acknowledge the learning resources provided by Google Developers and various researchers whose published works guided my methodology.

Finally, I am thankful to my family and friends for their continuous support and motivation, which helped me stay focused and complete this project on time.

Susan Aryal

LC ID: LC00017002100

Date: 2/20/2026

TABLE OF CONTENTS

DECLARATION OF AUTHENTICITY	ii
ACKNOWLEDGEMENTS	iii
TABLE OF FIGURES	ix
LIST OF FIGURES	x
ABBREVIATIONS AND ACRONYMS	xi
EXECUTIVE SUMMARY	xii
1 INTRODUCTION.....	1
1.1 Background of Study	1
1.2 Problem Statement	2
1.3 Objectives of the Study	3
1.3.1 General Objective.....	3
1.3.2 Specific Objectives	3
1.4 Scope of the Study.....	4
1.5 Significance of the Study.....	5
1.6 Limitations of the Study	6
1.7 Organization of the Report.....	6
2 LITERATURE REVIEW	8
2.1 Introduction.....	8
2.2 Review of Related Works	8
2.3 Theoretical Framework	10
2.4 Conceptual Framework	11
2.4.1 Identification of Variables.....	11
2.4.2 Framework Diagram and Relationship Analysis	11
2.5 Research Gap	12
2.6 Summary	13
3 METHODOLOGY	15
3.1 Research/Development Method.....	15
3.1.1 The Prototype Incremental Agile Framework.....	15

3.1.2	Research Design: Experimental Method.....	16
3.1.3	System Validation and Workflow	17
3.2	System Requirement Specification (SRS).....	17
3.2.1	Functional Requirements	17
3.2.2	Non-Functional Requirements	18
3.3	Tools and Technologies Used	19
3.3.1	Programming Languages	19
3.3.2	Machine Learning and AI Frameworks	19
3.3.3	Backend and Application Layer	19
3.3.4	Frontend and Presentation Layer.....	20
3.3.5	Development Tools	20
3.4	Feasibility Study	20
3.4.1	Technical Feasibility	20
3.4.2	Operational Feasibility	21
3.4.3	Economic Feasibility	21
3.5	System Architecture.....	22
3.5.1	Presentation Tier (Frontend).....	22
3.5.2	Logic Tier (Backend)	22
3.5.3	Data Tier (Database)	23
3.5.4	Synchronous Data Flow	23
4	SYSTEM DESIGN	24
4.1	Use Case Diagram & Description	24
4.1.1	Use Case Diagram	24
4.1.2	Detailed Use Case Descriptions	25
4.1.3	Actors.....	29
4.1.4	Operational Flow.....	29
4.2	Data Flow Diagram (DFD)	29
4.2.1	Level 0 DFD (Context Diagram).....	29
4.2.2	Level 1 DFD (Process Overview)	30

4.2.3	Level 2 DFD (Deep Dive into AI Pipeline).....	32
4.3	ER Diagram & Database Schema.....	33
4.3.1	Entity Relationship Diagram (ERD)	33
4.3.2	Database Schema (Physical Model).....	34
4.3.3	Relational Logic & Integrity	37
4.4	Class Diagram	37
4.4.1	Architectural Overview	37
4.4.2	Class Definitions	38
4.4.3	Class Relationships and Logic	39
4.5	Sequence & Activity Diagrams.....	40
4.5.1	Sequence Diagram: AI Analysis Workflow	40
4.5.2	Activity Diagram: Diagnostic Process	40
4.5.3	Synchronization Logic	41
4.6	UI/UX Design	42
4.6.1	Design Philosophy & Visual Identity	42
4.6.2	Layout Architecture	42
4.6.3	Key UX Features.....	42
4.6.4	Responsive and Accessible Design.....	43
5	SYSTEM IMPLEMENTATION	44
5.1	Module Description	44
5.1.1	Frontend Interface Module (Next.js/React).....	44
5.1.2	API Gateway & Controller Module (FastAPI)	44
5.1.3	AI Processing & Explainability Module (TensorFlow/Keras)	45
5.1.4	Data Persistence Module (PostgreSQL/SQLAlchemy)	45
5.1.5	Implementation Summary Table	45
5.2	Frontend Implementation.....	46
5.2.1	Component Architecture	46
5.2.2	State Management and Data Flow	47
5.2.3	Styling and User Experience (UX)	47

5.2.4	Client-Side PDF Generation.....	48
5.3	Backend Implementation	48
5.3.1	API Lifecycle & Request Handling	48
5.3.2	Object-Relational Mapping (ORM).....	49
5.3.3	Implementation Summary Table	49
5.3.4	Security and Optimization	50
5.4	Database Implementation	50
5.4.1	Relational Architecture.....	50
5.4.2	Object-Relational Mapping (ORM).....	50
5.4.3	Data Integrity & Persistence.....	51
5.5	Algorithms Used & AI Model Integration	51
5.5.1	The CNN Algorithm	51
5.5.2	Explainability: Grad-CAM Algorithm	52
5.5.3	Model Integration with FastAPI.....	52
5.5.4	Algorithm Summary Table.....	53
5.6	Screenshots of System Features	54
5.6.1	Dashboard & Case Registration	54
5.6.2	AI Analysis & Results Page	54
5.6.3	Visual Explainability (Grad-CAM).....	55
5.6.4	Diagnostic History & Records	56
6	CHAPTER 6: TESTING AND RESULT ANALYSIS	58
6.1	6.1 Test Plan	58
6.1.1	6.1.1 Testing Objectives	58
6.1.2	6.1.2 Testing Environment	58
6.1.3	6.1.3 Testing Metrics (Quantitative)	59
6.1.4	Qualitative Testing (Expert Review).....	59
6.2	Test Cases	59
6.2.1	Functional and Integration Test Cases	60
6.2.2	Performance & Boundary Testing	61

6.2.3	Test Result Summary	62
6.3	Unit Testing.....	62
6.3.1	Backend Unit Tests (Python/FastAPI).....	63
6.3.2	Frontend Unit Tests (React/Next.js).....	64
6.3.3	Testing Isolation (Mocking)	64
6.4	Integration Testing.....	64
6.4.1	Client-Server Communication	65
6.4.2	AI Model and Explainability Integration	65
6.4.3	Backend and Database (ORM) Integration	65
6.4.4	Integration Test Results Summary	66
6.5	System Testing.....	66
6.5.1	End-to-End Functional Flow	66
6.5.2	Performance and Latency Testing	67
6.5.3	Error Handling and Robustness	67
6.5.4	User Acceptance (UAT) Simulation.....	67
6.6	Performance Evaluation	68
6.6.1	Training History Analysis	68
6.6.2	Confusion Matrix Results	68
6.6.3	Quantitative Performance Metrics	69
6.6.4	Visual Verification (Sample Results)	70
7	CONCLUSION AND RECOMMENDATIONS.....	71
7.1	Summary	71
7.2	Conclusion	72
7.3	Limitations.....	72
7.4	Future Recommendations	73
8	REFERENCES	75
9	APPENDICES.....	76

TABLE OF FIGURES

Figure 2.1 Conceptual Framework	12
Figure 3.1 Gann Chart.....	16
Figure 3.2 Data Flow Diagram	23
Figure 4.1 Use Case Diagram	24
Figure 4.2 Level 0 DFD	30
Figure 4.3 Level 1 DFD	31
Figure 4.4 Level 2 DFD	33
Figure 4.5 Entity Relationship Diagram	34
Figure 4.6 Architectural Overview	38
Figure 4.7 Sequence & Activity Diagrams	41
Figure 5.1 State Management and Data Flow.....	47
Figure 5.2 Dashboard & Case Registration	54
Figure 5.3 AI Analysis & Results Page.....	55
Figure 5.4 Visual Explainability (Grad-CAM).....	56
Figure 5.5 Diagnostic History & Records.....	57

LIST OF FIGURES

Table 4.1 Use Case Description UC-01	25
Table 4.2 Use case description UC-02	26
Table 4.3 Use Case Description UC-03	28
Table 4.4 Database Schema Users	34
Table 4.5 Database Schema Scans	35
Table 4.6 Class Definitions	38
Table 4.7 UI Features	42
Table 5.1 Frontend Implementation Technologies	45
Table 5.2 Backend Implementation Technology	49
Table 5.3 AI Algorithm Summary	53
Table 6.1 Model Testing Matrix	59
Table 6.2 Functional and Integration Test Case	60
Table 6.3 Performance and Boundary Testing Case	61
Table 6.4 Unit Testing Backend	63
Table 6.5 Summary of Integration Test	66
Table 6.6 Error Scenario	67
Table 6.7 Model Performance Matrix	69

ABBREVIATIONS AND ACRONYMS

The following list provides the definitions for the technical terms and abbreviations used throughout this project report:

- AI: Artificial Intelligence
- ANN: Artificial Neural Network
- API: Application Programming Interface
- CNN: Convolutional Neural Network
- DL: Deep Learning
- F1-Score: The harmonic mean of precision and recall
- GPU: Graphics Processing Unit
- IDE: Integrated Development Environment (e.g., VS Code, PyCharm)
- Keras: An open-source software library that provides a Python interface for artificial neural networks
- ML: Machine Learning
- MRI: Magnetic Resonance Imaging
- ReLU: Rectified Linear Unit (Activation Function)
- RGB: Red, Green, Blue (Color Model)
- SVM: Support Vector Machine
- TensorFlow: An end-to-end open-source platform for machine learning

EXECUTIVE SUMMARY

The primary objective of this project is to develop an automated system for the detection and multi-class classification of brain tumors using advanced machine learning techniques. Brain tumors represent a significant health challenge where early and precise diagnosis is a critical factor in determining patient treatment plans and survival rates. Traditional manual interpretation of Magnetic Resonance Imaging (MRI) scans by radiologists is often time-consuming and prone to human error or subjectivity. This study addresses these limitations by implementing a Deep Learning approach, specifically utilizing a Convolutional Neural Network (CNN) architecture designed to categorize MRI slices into four distinct classes: Glioma, Meningioma, Pituitary tumor, and No Tumor.

The methodology involved the use of a comprehensive dataset comprising thousands of MRI images, which underwent rigorous preprocessing including resizing, normalization, and data augmentation to ensure model robustness. By leveraging the Keras and TensorFlow frameworks, the model was trained to automatically extract complex spatial features from the medical images without the need for manual feature engineering. The experimental results demonstrated a strong overall performance with a classification accuracy of 86%. Notably, the system achieved a 97% recall rate for Pituitary tumors and a high F1-score of 0.92 for scans with no tumors, indicating its reliability as a preliminary screening tool.

In conclusion, this project successfully demonstrates that deep learning models can serve as effective decision-support systems in clinical radiology. While the model shows high precision in identifying Gliomas and excellent sensitivity for Pituitary tumors, future work could involve the integration of transfer learning and larger, multi-institutional datasets to further refine diagnostic accuracy across all tumor types. This automated tool holds the potential to reduce the diagnostic workload for medical professionals and provide a faster, secondary layer of verification in the neuro-oncology workflow.

1 INTRODUCTION

1.1 Background of Study

In the contemporary medical landscape, neurological disorders represent a significant portion of global health challenges, with brain tumors being among the most life-threatening conditions. A brain tumor is defined by an abnormal growth of cells within the brain, which can be either benign or malignant, making accurate and timely diagnosis the cornerstone of effective clinical intervention. Currently, Magnetic Resonance Imaging (MRI) is the gold standard for neuro-imaging due to its non-invasive nature and its superior ability to provide high-contrast images of soft tissues, allowing for the visualization of complex brain structures. Despite these technological advancements in imaging, the interpretation of MRI scans remains a largely manual and labor-intensive process performed by radiologists, which introduces several critical bottlenecks into the diagnostic workflow.

The manual approach to MRI analysis is inherently susceptible to human fatigue and subjectivity, particularly in modern hospitals where the high volume of daily scans can lead to diagnostic exhaustion and an increased risk of error. Furthermore, the visual similarity between different tumor types, such as Glioma and Meningioma, often presents overlapping morphological features that make it difficult for even experienced practitioners to differentiate them through visual inspection alone. This complexity is compounded by a global scarcity of expertise, where many regions suffer from an insufficient ratio of specialized radiologists to patients, resulting in significant delays in reporting and subsequent treatment.

To address these systemic challenges, the field of Medical Informatics has increasingly turned toward Artificial Intelligence (AI) and Machine Learning (ML). Specifically, Deep Learning (DL) through Convolutional Neural Networks (CNNs) has demonstrated exceptional capabilities in pattern recognition and image classification. Unlike traditional computer-aided diagnosis which relies on hand-crafted features, CNNs can automatically extract intricate hierarchical features from raw pixels, often identifying subtle textures and anomalies that are imperceptible to the human eye. This automation not only promises to

standardize the diagnostic process but also significantly reduces the time required for initial screenings.

The NeuroScan AI project is conceived to bridge the gap between advanced deep learning research and practical clinical utility by developing a full-stack automated system capable of categorizing MRI scans into four distinct classes: Glioma, Meningioma, Pituitary, and No Tumor. To overcome the "black-box" nature of traditional AI which often provides a result without justification this study integrates Gradient-weighted Class Activation Mapping (Grad-CAM). This technique provides visual explainability by generating heatmaps that highlight the specific anatomical regions the model prioritized during its classification process, thereby fostering clinical trust. By integrating a high-performance FastAPI backend with a modern Next.js frontend, this project aims to provide a seamless, end-to-end diagnostic tool that empowers medical professionals with rapid, reliable, and interpretable second opinions.

1.2 Problem Statement

The fundamental challenge addressed by this research is the critical reliance on manual diagnostic processes in neuro-oncology, which are inherently prone to human error and significant delays. While Magnetic Resonance Imaging (MRI) serves as a primary tool for detecting brain abnormalities, the sheer volume of data produced in clinical settings often overwhelms the available radiological expertise. This bottleneck is compounded by the high degree of visual similarity between various tumor types, specifically Glioma and Meningioma, which can lead to inter-observer variability and misdiagnosis. Our preliminary evaluation of the current model reinforces this concern, as evidenced by a lower recall rate for Glioma compared to other categories, illustrating that even computational models face difficulty in distinguishing these overlapping morphological features.

Furthermore, a significant gap exists between high-accuracy artificial intelligence research and clinical trust. Most existing automated systems operate as "black boxes," providing a classification result without any visual justification for the decision. In a medical environment, clinicians are hesitant to rely on an algorithm that does not transparently display its "Region of Interest." There is a distinct lack of integrated IT solutions that

combine a high-performance backend, a modern user interface, and explainable AI techniques like Grad-CAM. Consequently, there is an urgent need for a system that not only classifies brain tumors with high precision but also provides visual heatmaps to justify their findings, thereby ensuring that the diagnosis is both rapid and verifiable by a medical professional.

1.3 Objectives of the Study

The primary ambition of this research is to bridge the gap between complex deep learning architectures and practical, trust-based clinical applications by developing a robust, end-to-end diagnostic support system. The objectives are divided into a broad overarching goal and the technical milestones required to reach it.

1.3.1 General Objective

To develop a transparent and user-friendly AI system for detecting and classifying brain tumors from MRI images with interpretable visual explanations that assist medical professionals in making faster and more accurate clinical decisions.

1.3.2 Specific Objectives

To achieve the general objective, the following technical and functional goals have been established:

- To design and train a custom Convolutional Neural Network (CNN) to automatically detect the presence of a brain tumor in MRI images and classify it into one of four categories: Glioma, Meningioma, Pituitary Tumor, or No Tumor.
- To apply Grad-CAM (Gradient-weighted Class Activation Mapping) visualization to highlight tumor regions influencing the model's predictions, thereby improving interpretability for clinical validation.
- To develop and deploy a secure three-tier web application using FastAPI for the backend and Next.js for the frontend to facilitate MRI upload, real-time detection, and result visualization.

- To implement a prediction history management system that allows clinicians to store and retrieve previous analysis results associated with specific patient metadata.
- To integrate an automated reporting feature that generates downloadable PDF summaries containing the AI diagnosis and the generated heatmap for medical documentation.

1.4 Scope of the Study

The scope of this project is meticulously defined to encompass the development of a comprehensive, clinical-grade software solution for the categorization and visualization of brain tumors. It specifically includes the identification of three primary tumor types Glioma, Meningioma, and Pituitary alongside a "No Tumor" category to represent healthy brain tissue. The technical scope spans the entire software development lifecycle, integrating a custom-trained Convolutional Neural Network (CNN) with a high-performance FastAPI backend and a responsive Next.js frontend dashboard. The system is designed to provide real-time inference, generating both a numerical probability of the tumor type and a visual activation map using Grad-CAM. Furthermore, the scope includes a patient management interface that allows for the capture of clinical metadata and the generation of standardized, downloadable PDF reports. These reports are intended to serve as a bridge between AI-generated insights and traditional medical documentation.

Despite the comprehensive nature of the system, certain functional and academic limitations must be acknowledged to define its operational boundaries. The research is primarily constrained by the characteristics of the training dataset, meaning the model is optimized for T1-weighted MRI images and may not demonstrate the same level of accuracy with other imaging modalities such as CT scans or PET scans. Academically, the project serves as a diagnostic support tool and is not intended to replace the professional judgment of a neuro-radiologist or neurosurgeon. Furthermore, the model's predictive power is limited to the four specific classes it was trained on; it does not currently account for rarer brain pathologies such as secondary metastatic tumors, abscesses, or strokes. Finally, while the system is architected for clinical use, its current performance is subject

to the computational limits of local GPU resources, which dictated the depth and iteration count of the neural network during the training phase.

1.5 Significance of the Study

The significance of this study is multifaceted, offering profound benefits across academic, practical, and societal domains. By bridging the gap between advanced deep learning and clinical practice, the project provides a tangible solution to the limitations of manual MRI interpretation.

From an academic and technical perspective, this project contributes significantly to the field of Explainable Artificial Intelligence (XAI) in medical informatics. It addresses the prevailing "black box" criticism of deep learning models by demonstrating how Gradient-weighted Class Activation Mapping (Grad-CAM) can be successfully integrated into a production-level full-stack environment. By documenting the synergy between a high-performance FastAPI backend and a modern Next.js frontend, this research provides a technical blueprint for future developers and researchers aiming to build interoperable, transparent, and high-speed medical diagnostic software. It proves that neural networks can be made verifiable, fostering a new standard for trust in machine-human collaboration within healthcare IT.

Practically, NeuroScan AI serves as a critical decision-support tool that enhances the operational efficiency of radiological departments. For medical professionals, the system acts as an intelligent force multiplier, reducing the cognitive burden of screening high volumes of MRI data. By automating the initial classification and highlighting suspicious regions via heatmaps, it allows radiologists to prioritize complex cases and focus their attention on critical areas with higher precision, thereby minimizing the risk of diagnostic fatigue and human error. The automated reporting feature further streamlines clinical workflows by providing standardized, data-driven documentation that can be easily integrated into a patient's digital health record.

From a societal standpoint, the deployment of such a system has the potential to facilitate earlier medical intervention. In the context of neuro-oncology, where early detection is the most significant factor in patient survival rates, reducing the "search and find" time for

tumors is vital. By providing a rapid and reliable second opinion, the project helps in making advanced neurological screening more accessible, ultimately contributing to better patient outcomes and the overall advancement of digital health infrastructure.

1.6 Limitations of the Study

Despite the comprehensive nature of the NeuroScan AI system, several functional and academic limitations must be acknowledged to define its operational boundaries. The primary constraint of this research is its heavy reliance on the characteristics of the training dataset; the current model is strictly optimized for T1-weighted MRI images and may not demonstrate the same level of diagnostic accuracy when processing other imaging modalities, such as CT scans, PET scans, or T2-weighted sequences. Furthermore, while the system is designed to identify the three most common primary brain tumors Glioma, Meningioma, and Pituitary it does not currently account for rarer neurological pathologies, including secondary metastatic tumors, cerebral abscesses, or strokes.

From a clinical and legal perspective, this project is developed as a diagnostic support tool and is not intended to function as a replacement for the professional judgment of a board-certified neuro-radiologist or neurosurgeon. The AI-generated heatmaps and classification probabilities serve as a "second opinion" to assist in screening, but they do not constitute a final medical diagnosis. Technically, the system's performance is also subject to the computational limits of the local hardware used during the training phase. Due to the high memory requirements of deep neural networks, the architecture was optimized based on existing GPU resources, which inherently limited the depth of the model and the number of training iterations that could be performed within the project timeline. Finally, as the system is a web-based prototype, its real-time performance is dependent on stable internet connectivity and the processing overhead of the hosting environment.

1.7 Organization of the Report

This research report is structured into several interconnected chapters that systematically document the development of the NeuroScan AI system, from its conceptual inception to its final technical evaluation. Chapter 1: Introduction establishes the foundation of the study by detailing the background, problem statement, and objectives, while also defining

the scope and limitations of the project to set realistic expectations for the proposed solution.

Following the introduction, Chapter 2: Literature Review provides a rigorous academic analysis of existing research in the domains of neuro-oncology and deep learning. This chapter evaluates the evolution of Convolutional Neural Networks (CNNs), the current state of brain tumor classification technologies, and the critical role of Explainable AI (XAI) through tools like Grad-CAM. Chapter 3: System Analysis and Research Methodology transitions into the technical planning phase, outlining the Agile development lifecycle, requirement gathering, and the feasibility study conducted to ensure the project's viability.

The technical core of the report is contained within Chapter 4: System Design and Chapter 5: Implementation and Testing. Chapter 4 details the architectural framework of the system, including the three-tier architecture (FastAPI, Next.js, and the CNN model), data flow diagrams, and UI/UX design principles. Chapter 5 focuses on the actual coding process, model training iterations, and the comprehensive testing strategies unit, integration, and user acceptance testing employed to verify system stability. Finally, Chapter 6: Conclusion and Recommendations summarizes the research findings, reflects on the project's achievements against its initial objectives, and suggests future enhancements for the system. The report concludes with the References and Appendices, containing the source code snippets and supplemental data used throughout the study.

2 LITERATURE REVIEW

2.1 Introduction

Literature review serves as the critical theoretical foundation for this research, providing a comprehensive analysis of the existing body of knowledge regarding brain tumor classification and the evolution of medical imaging technologies. As neurological disorders continue to pose a significant threat to global health, the intersection of Medical Informatics and Artificial Intelligence has become a focal point for academic and clinical innovation. This chapter evaluates the transition from manual radiological interpretation to automated deep learning frameworks, specifically focusing on the efficacy of Convolutional Neural Networks (CNNs) in identifying pathological patterns within Magnetic Resonance Imaging (MRI) data.

Furthermore, this section explores the persistent challenge of the "black-box" nature of deep learning models, which often hinders their adoption in clinical settings due to a lack of transparency. By reviewing contemporary studies on Explainable Artificial Intelligence (XAI), this chapter establishes the necessity of interpretability tools like Gradient-weighted Class Activation Mapping (Grad-CAM). The review is structured to first address the medical classifications of brain tumors, followed by a comparative analysis of various neural network architectures, and finally, an examination of current full-stack implementations in the healthcare domain. Through this systematic investigation, the project identifies existing research gaps that NeuroScan AI aims to address, particularly the integration of high-accuracy classification with real-time, verifiable visual evidence.

2.2 Review of Related Works

The automation of brain tumor classification has been a focal point of medical informatics for over a decade, with researchers continuously striving to bridge the gap between high accuracy and clinical utility. A significant milestone in this domain was established by Deepak and Ameer (2023), who implemented a transfer learning approach using the GoogLeNet architecture. By applying rigorous normalization and resizing techniques to a dataset of 3,064 MRI images, they achieved a remarkable classification accuracy of 98%. Their work proved instrumental in demonstrating that pre-trained models, originally

developed for general image recognition, could be effectively repurposed to identify intricate hierarchical features within relatively small medical datasets. This success laid the groundwork for further exploration into how deep architecture can be optimized for the specific textures of neurological tissue.

Building upon these architectural foundations, researchers have increasingly recognized that pure accuracy is insufficient for clinical adoption without a corresponding level of transparency. Hussain and Shouno (2023) addressed this critical need for explainability in neuro-oncology by utilizing a multiclass Convolutional Neural Network (CNN) combined with Gradient-weighted Class Activation Mapping (Grad-CAM). Their study focused not only on the classification of tumors but also on their precise localization within the cranial cavity. This research was pivotal in proving that visual heatmaps could successfully align with established radiological landmarks, thereby increasing the clinical transparency required for physicians to trust and adopt AI-driven diagnostic tools. By providing a "Region of Interest," their methodology bridged the gap between raw data output and human-verifiable evidence.

The complexity of the diagnostic task has also led to significant innovations in image preprocessing and comparative model analysis. Recent work by Putra et al. (2025) introduced an enhanced deep learning framework that integrated Contrast Limited Adaptive Histogram Equalization (CLAHE) for preprocessing. Their fine-tuned model achieved an accuracy of 94.2% on a dataset of 3,000 images, emphasizing that sophisticated image enhancement techniques are vital for reducing noise and improving the recall rates of subtle tumor boundaries that are often missed in raw scans. Simultaneously, Bukaita and Vadde (2025) conducted a comparative evaluation between custom-built CNNs and the ResNet18 architecture. Their findings revealed that while custom models offer greater computational efficiency for edge devices, residual architectures like ResNet18 achieve superior performance, reaching an accuracy of 99.7%. This superiority is largely attributed to the residual connections which solve the vanishing gradient problem in deep networks, allowing for the extraction of the highly complex features necessary for distinguishing between Glioma and Meningioma.

2.3 Theoretical Framework

The theoretical foundation of this study is anchored in the synergy between Deep Learning (DL) and Computer Vision, specifically focusing on the automated extraction of pathological features from complex medical imagery. At the core of this framework is the Convolutional Neural Network (CNN), which mimics the human visual cortex by processing data through a hierarchy of layers. Unlike traditional machine learning, which requires manual feature engineering, CNNs utilize convolutional layers where mathematical kernels scan the input data to identify spatial patterns such as edges, textures, and shapes. These patterns are further processed through pooling layers to ensure translation invariance and Rectified Linear Units (ReLU) to introduce non-linearity, allowing the model to learn the complex, non-linear boundaries that characterize different brain tumor types.

To enhance the model's performance on specialized medical data, the framework leverages the theory of Transfer Learning. This methodology is based on the premise that knowledge gained while solving one problem can be applied to a different but related task. By utilizing architectures pre-trained on large-scale datasets like ImageNet, the system inherits established feature weights that are already optimized for recognizing general shapes and gradients. Fine-tuning these pre-trained networks for neuro-oncology significantly reduces the computational power and the massive data volume typically required to train deep networks from scratch. This theoretical approach is crucial in the medical domain, where high-quality, labeled datasets are often limited, yet high diagnostic sensitivity and specificity are mandatory.

Furthermore, the research incorporates the theory of Explainable Artificial Intelligence (XAI) through the implementation of Gradient-weighted Class Activation Mapping (Grad-CAM). The theoretical basis of Grad-CAM involves using the gradients of any target concept (such as a specific tumor class) flowing into the final convolutional layer to produce a localization map. By computing neuron importance weights, the system generates a visual heatmap that highlights the specific regions within the MRI scan that most heavily influenced the model's prediction. This theoretical addition is vital for clinical transparency, transforming the system from an opaque "black box" into a verifiable

diagnostic tool that allows medical professionals to correlate AI predictions with anatomical reality.

2.4 Conceptual Framework

The conceptual framework for NeuroScan AI serves as the structural foundation of the research, illustrating how specific technical inputs are transformed into diagnostic outcomes. This framework acts as a logical bridge between the theoretical principles of deep learning and the practical implementation of the three-tier system architecture. By identifying the clear boundaries and interactions between the independent and dependent variables, the framework provides a verifiable roadmap for achieving high-accuracy brain tumor classification.

2.4.1 Identification of Variables

The research identifies a specific set of variables that dictate the system's performance and clinical utility:

- **Independent Variables:** These are the factors that influence the final classification result and are manipulated or provided as inputs to the system. They include the CNN Model Architecture (layer configuration and hyperparameters), Image Preprocessing Techniques (normalization and resizing), and MRI Image Characteristics (tumor type and scan quality).
- **Dependent Variable:** The primary outcome is the Brain Tumor Classification Result. This represents the final categorization of the input image into one of four classes: Glioma, Meningioma, Pituitary Tumor, or No Tumor. The accuracy and reliability of this dependent variable are directly contingent upon the interaction of the independent inputs.

2.4.2 Framework Diagram and Relationship Analysis

The relationship between these variables is visually represented in the framework diagram below, generated using professional data visualization tools to reflect the system's internal logic.

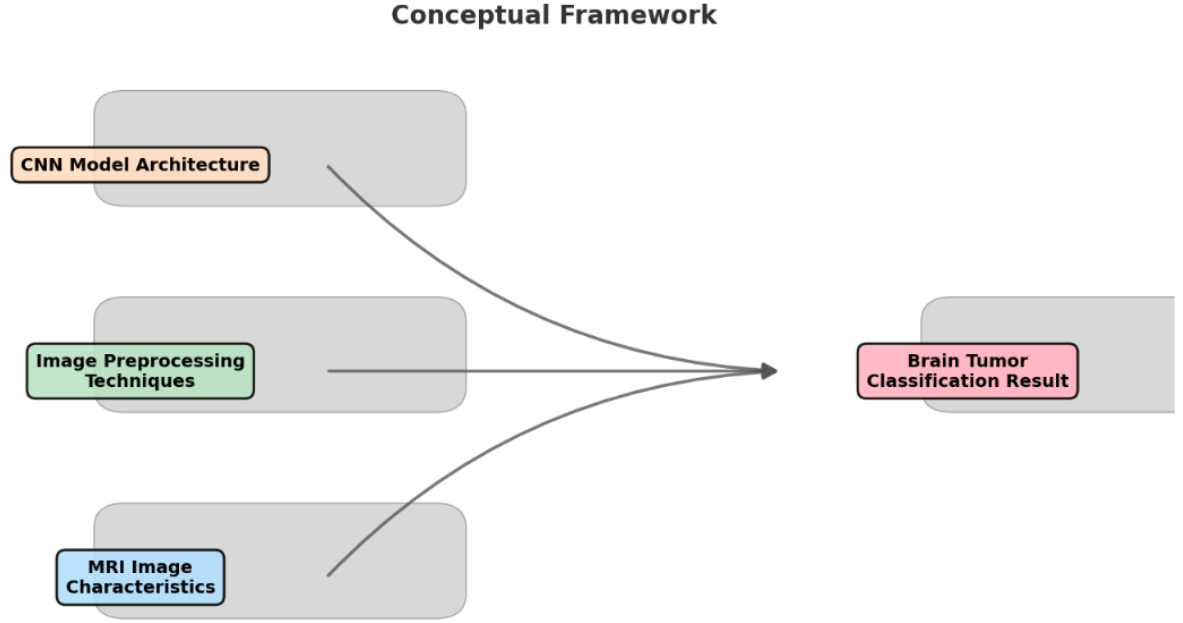


Figure 2.1 Conceptual Framework

As illustrated in Figure 2.1, the framework follows a Process Flow model where the independent variables converge to produce the final outcome. The process begins with the raw MRI Image Characteristics, which serve as the primary data source. These images are subjected to a series of Image Preprocessing Techniques such as normalization to standard pixel ranges and resizing to fixed dimensions to ensure the input is compatible with the model. These refined inputs are then processed by the CNN Model Architecture, where deep layers extract the hierarchical features necessary for identification. The synergy between these factors results in the Classification Result, which is the final dependent output of the system. This structured relationship ensures that any improvement in the independent variables, such as a more optimized learning rate or better contrast adjustment, results in a corresponding enhancement in the diagnostic accuracy of the final classification.

2.5 Research Gap

Despite the significant advancements in deep learning for neuro-oncology, a critical evaluation of emergent literature between 2023 and 2026 reveals a persistent "translation gap" between laboratory success and clinical utility. While contemporary models

frequently achieve state-of-the-art accuracies exceeding 98%, their practical implementation remains limited by several fundamental deficiencies in current research methodologies. A primary concern is the fragmentation of diagnostic workflows, as a significant majority of existing research treats brain tumor classification as an isolated mathematical problem. Most studies focus exclusively on optimizing CNN architectures on static datasets but fail to provide an end-to-end framework, leaving a clear lack of integrated, three-tier full-stack systems that facilitate a seamless journey from raw MRI upload to secure cloud-based retrieval.

Furthermore, the "Explainability-Trust Deficit" continues to hinder adoption; although high-performance networks like ResNet provide precise labels, they are often criticized as "black boxes" that lack clinical justification. While theoretical work on Grad-CAM exists, it is rarely integrated into live web platforms as a real-time diagnostic feature, leaving a gap where clinicians are expected to trust AI predictions without seeing the visual heatmaps that correlate the AI's "Region of Interest" with anatomical landmarks. This issue is compounded by the absence of standardized clinical documentation, as most AI models provide only a digital score rather than a structured record. There is a notable research gap in systems that automatically convert findings into downloadable, standardized PDF reports which are essential for archiving and multi-disciplinary team meetings. Finally, the project addresses the widespread issue of preprocessing inconsistency across protocols; because medical images vary by scanner brand, this study bridges the gap by implementing a standardized preprocessing pipeline within the logic tier to ensure model robustness across heterogeneous, real-world MRI sources.

The following summary synthesizes the primary findings derived from the literature and establishes the empirical grounding for the proposed research.

2.6 Summary

The research derived from the literature indicates that while Deep Learning, specifically Convolutional Neural Networks (CNNs) and Transfer Learning, has reached a high level of technical maturity, a significant gap remains in transitioning these models from experimental code to functional medical tools. A critical finding is that high diagnostic accuracy (often exceeding 95% in recent 2025 studies) is no longer the sole metric for

success; instead, the focus has shifted toward Explainable AI (XAI). The integration of tools like Grad-CAM is now recognized as essential for bridging the "trust deficit" between automated systems and clinical practitioners by providing visual evidence for every classification.

Furthermore, the research confirms that the performance of these models is heavily dependent on the quality of the independent variables, specifically the preprocessing pipeline. Standardizing "real-world" MRI data through normalization and resizing is identified as a vital step to ensure model robustness across different imaging protocols and scanner brands. Finally, the evidence suggests that current solutions are often fragmented, lacking the full-stack integration necessary to handle secure data flow, real-time processing, and automated reporting. These findings collectively justify the development of a unified system that combines high-performance neural networks with a transparent, user-centric architecture and standardized clinical documentation.

3 METHODOLOGY

3.1 Research/Development Method

The development of NeuroScan AI (referred to in the synopsis as the "AI-Based MRI Image Classification for Brain Tumor Categorization" system) follows a Prototype Incremental Agile development model. This hybrid approach combines early prototyping with structured, incremental cycles to reduce development risks and allow for continuous refinement of the system's deep learning capabilities.

3.1.1 *The Prototype Incremental Agile Framework*

This model is particularly suited for medical AI projects as it enables iterative improvement of the CNN model alongside the progressive integration of explainability (Grad-CAM) and full-stack deployment components. The development process is broken down into seven distinct functional increments:

- Increment 1: Data Pipeline Construction – Establishing the dataset collection and preprocessing workflow, including resizing, normalization, and augmentation.
- Increment 2: Binary Prototype – Developing an initial prototype CNN designed for simple binary classification (Tumor vs. No Tumor).
- Increment 3: Multi-Class Expansion – Scaling the CNN architecture to categorize four specific classes: Glioma, Meningioma, Pituitary tumor, and No Tumor.
- Increment 4: Explainability Integration – Integrating the Grad-CAM module to generate visual heatmaps for prediction transparency.
- Increment 5: Logic Tier Development – Building the FastAPI backend and configuring the PostgreSQL database for secure data persistence.
- Increment 6: Presentation Tier Development – Developing the user-facing interface using Next.js.
- Increment 7: Finalization – Conducting comprehensive system testing, deployment, and final documentation.

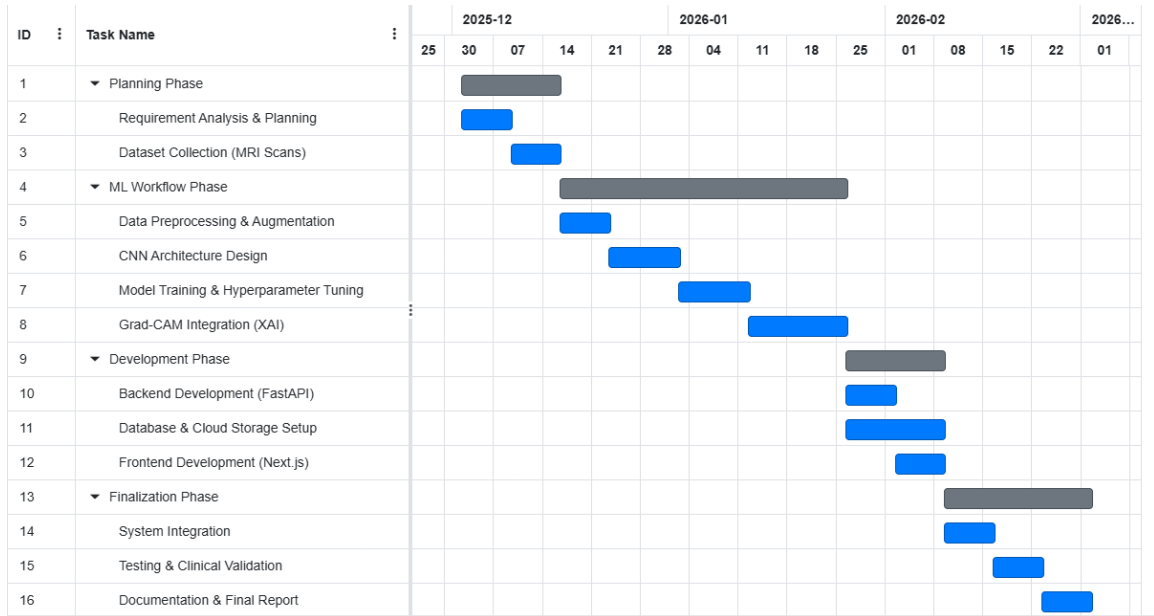


Figure 3.1 Gann Chart

3.1.2 Research Design: Experimental Method

From a research perspective, this project utilizes an Experimental Design to systematically evaluate the performance of the custom CNN. This design is structured to observe how changes in system inputs affect the final diagnostic output:

- **Independent Variables:** These are the controlled technical factors, including the Custom CNN Architecture (number of convolutional and pooling layers), the Adam Optimizer settings, and Preprocessing Parameters (such as resizing images to 224x224 pixels and pixel normalization).
- **Dependent Variables:** The outcome of the research is measured through the Classification Results, specifically evaluated using standard metrics: Accuracy, Precision, Recall, F1-score, and the Confusion Matrix.
- **Causal Relationship:** By keeping the dataset constant and iteratively adjusting the CNN layers (the independent variables), the research can pinpoint the most effective configuration for reducing false negatives a critical requirement in clinical neuro-imaging.

3.1.3 System Validation and Workflow

Each increment produces a functional prototype that undergoes rigorous validation before proceeding to the next stage. The final system integrates these researched components into a unified Three-Tier Architecture, ensuring that the experimental AI engine is accessible through a practical, secure, and user-friendly web platform.

3.2 System Requirement Specification (SRS)

The SRS outlines the comprehensive functional capabilities and quality constraints required for the successful deployment of NeuroScan AI. These requirements ensure that the system not only classifies tumors accurately but also adheres to the security and usability standards necessary for medical software.

3.2.1 Functional Requirements

Functional requirements define the specific behaviors and services that the system must perform.

- FR-1: User Access Control: The system must implement a secure login module for radiologists and researchers to protect patient confidentiality and restrict system access to authorized personnel.
- FR-2: Multi-Format Image Ingestion: The application must support the upload of MRI images in standard web formats (JPG, PNG) and medical-grade NIFTI (.nii) files.
- FR-3: Automated Image Normalization: Upon upload, the system must execute an automated preprocessing pipeline that resizes images to 224x224 pixels and applies pixel-value normalization for model compatibility.
- FR-4: 4-Class Tumor Categorization: The custom CNN must analyze the input scan and provide a classification into one of four categories: Glioma, Meningioma, Pituitary tumor, or No Tumor, accompanied by a probability confidence score.
- FR-5: Explainability Visualization (Grad-CAM): For every positive tumor prediction, the system shall generate a visual heatmap (Grad-CAM) that overlays the MRI to highlight the anatomical regions used by the AI for its decision.

- FR-6: Persistent Case History: The system must store every prediction, including the original image and the Grad-CAM visualization, in a PostgreSQL database for retrospective clinical review.
- FR-7: Clinical Report Generation: The system should provide an interface to export classification results and heatmaps into a structured, downloadable document suitable for archiving or multidisciplinary team (MDT) meetings.

3.2.2 Non-Functional Requirements

Non-functional requirements describe the "quality attributes" and operational constraints of the system.

- NFR-1: Diagnostic Reliability: The AI model must be optimized to minimize false negatives through a high-sensitivity training approach, as misdiagnosing a present tumor carries high clinical risk.
- NFR-2: Latency and Response Time: The system must process an MRI scan including preprocessing, CNN inference, and heatmap generation within 5 seconds to maintain an efficient diagnostic workflow.
- NFR-3: Data Integrity and Security: All patient data and prediction results stored in the PostgreSQL database must be encrypted, and communication between the frontend and backend must occur over secure HTTPS protocols.
- NFR-4: Interface Intuition (Usability): The Next.js frontend must be designed for clarity, ensuring that clinicians can navigate from image upload to result visualization with fewer than three clicks.
- NFR-5: System Scalability: The three-tier architecture must allow for the independent scaling of the FastAPI backend to handle concurrent image processing requests without performance degradation.
- NFR-6: Hardware Accessibility: The system must be accessible via standard web browsers (Chrome, Edge, Safari), eliminating the need for radiologists to install specialized local hardware or software.

3.3 Tools and Technologies Used

The implementation of NeuroScan AI utilizes a modern, full-stack development environment designed to support computationally intensive AI tasks while providing a responsive and secure user experience. The technology stack is divided into three primary categories: Machine Learning, Backend (Logic Tier), and Frontend (Presentation Tier).

3.3.1 Programming Languages

- Python: Used as the primary language for backend development and machine learning due to its extensive library ecosystem for data science and AI.
- JavaScript (TypeScript): Employed for building the interactive frontend, ensuring a dynamic and responsive user interface.

3.3.2 Machine Learning and AI Frameworks

- TensorFlow / Keras: The core library used to design, train, and validate the custom Convolutional Neural Network (CNN) architecture.
- OpenCV: Utilized within the preprocessing module for image manipulation tasks such as resizing MRI scans to the required 224x224 pixel dimensions and performing pixel normalization.
- Matplotlib: Employed for generating the Grad-CAM heatmaps, which provide the visual justifications for the model's predictions.

3.3.3 Backend and Application Layer

- FastAPI: A high-performance Python web framework used to build the RESTful APIs. It manages the communication between the frontend and the AI model, handling image uploads and returning classification results.
- PostgreSQL: A robust relational database used to ensure secure and structured storage of MRI images, prediction results, Grad-CAM visualizations, and user case history.

3.3.4 Frontend and Presentation Layer

- Next.js: A React-based framework used to develop the user interface. It provides a seamless experience for radiologists to upload scans and view real-time diagnostic reports.

3.3.5 Development Tools

- Visual Studio Code / PyCharm: Used as the primary Integrated Development Environments (IDEs) for writing and debugging the system code.
- Jupyter Notebook: Employed during the research and experimentation phase for model prototyping, hyperparameter tuning, and data visualization.

3.4 Feasibility Study

The feasibility study evaluates the practicality and sustainability of NeuroScan AI across technical, operational, and economic dimensions. This assessment ensures that the project can be successfully completed and maintained within the defined constraints of the current research environment.

3.4.1 Technical Feasibility

The technical feasibility focuses on the availability of the required hardware and software to process complex neuro-imaging data.

- Deep Learning Frameworks: The project utilizes TensorFlow and Keras, which are industry-standard libraries capable of handling the custom CNN architecture required for tumor categorization.
- Hardware Compatibility: The research is supported by NVIDIA GPU acceleration (RTX series), which provides the computational power necessary for training the model and generating Grad-CAM heatmaps in real-time.
- Architecture Stability: The Three-Tier Architecture (Next.js, FastAPI, and PostgreSQL) is a proven web standard that ensures the system can scale and handle MRI data securely.

- **Data Availability:** The use of established, publicly available MRI datasets ensures that there is sufficient labeled data to train the model to a high degree of accuracy without the legal complexities of raw hospital data access.

3.4.2 Operational Feasibility

Operational feasibility assesses how well the system fits into the daily workflow of the target users.

- **User Intuition:** By utilizing Next.js for the frontend, the system provides a modern, responsive interface that requires minimal technical training for radiologists and medical students to operate.
- **Clinical Value:** The integration of Grad-CAM addresses the "black-box" problem found in prior research, providing the transparency and visual evidence that clinicians require to trust AI-assisted decisions.
- **Workflow Integration:** The system's ability to handle the end-to-end process from raw image upload to result history management ensures it functions as a practical tool rather than just a laboratory experiment.

3.4.3 Economic Feasibility

Economic feasibility analyzes the cost-effectiveness and resource requirements of the project.

- **Development Costs:** The project primarily utilizes Open-Source technologies (Python, FastAPI, PostgreSQL), significantly reducing software licensing expenses.
- **Infrastructure:** The system is designed to run on standard hardware and can be deployed via cloud-based platforms, making it an affordable solution for resource-limited healthcare settings.
- **Time Management:** The Prototype Incremental Agile model ensures that the project is completed within the allocated 90-day timeline, preventing costly development delays.

3.5 System Architecture

NeuroScan AI is built on a Three-Tier Architecture that operates within a localized environment. This design ensures that the system is modular, allowing the AI logic to function independently of the user interface while maintaining a direct, synchronous data flow for real-time diagnostics.

3.5.1 *Presentation Tier (Frontend)*

The Presentation Tier is the entry point for the user (Radiologist/Researcher). It is developed using Next.js to provide a highly responsive, single-page experience.

- Role: It manages the state of the user's session, handles the file selection for MRI scans, and renders the final diagnostic outputs.
- Key Components:
 - Dashboard: A unified interface for uploading new scans and viewing results.
 - Visualizer: A specialized canvas component that displays the original MRI side-by-side with the generated Grad-CAM heatmap.
 - Local Storage Interface: Fetches and displays the history of previous classifications directly from the backend.

3.5.2 *Logic Tier (Backend)*

The Logic Tier acts as the central coordinator, built with FastAPI. It handles all "heavy lifting," including image processing and AI inference.

- Role: It receives the raw image from the frontend, executes the processing pipeline, and returns the classification results.
- Key Components:
 - Inference Engine: Loads the trained CNN model to perform real-time classification (Glioma, Meningioma, Pituitary, or No Tumor).
 - XAI Module: Uses the gradient information from the final convolutional layer to produce a Grad-CAM heatmap.

- API Controllers: RESTful endpoints that manage the synchronous flow of data from the user to the database.

3.5.3 Data Tier (Database)

The Data Tier provides a persistent storage solution using PostgreSQL.

- Role: Unlike a simple script, this tier ensures that every diagnostic session is recorded and can be audited or reviewed later.
- Key Components:
 - Image Metadata Table: Stores the paths and timestamps of uploaded MRI scans.
 - Diagnosis Table: Records the model's classification results, confidence scores, and the generated heatmap references linked to each specific scan.

3.5.4 Synchronous Data Flow

The system follows a Request-Response cycle. When a user clicks "Analyze," the data travels from the Presentation Tier to the Logic Tier. The user remains on the screen while the backend performs preprocessing and inference. Once the database is updated with the result, the Logic Tier sends a success response back to the Frontend, which immediately updates the UI with the diagnosis and heatmap.

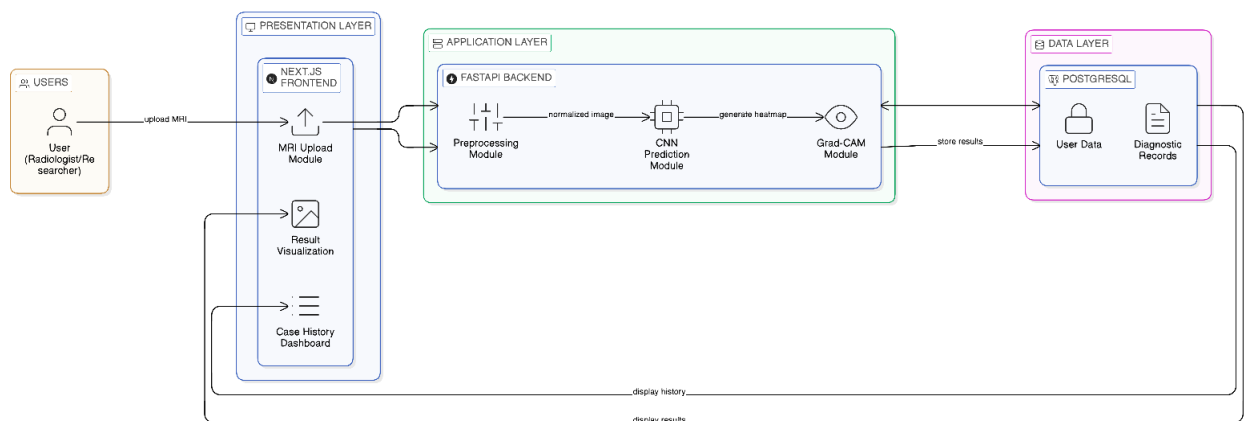


Figure 3.2 Data Flow Diagram

4 SYSTEM DESIGN

4.1 Use Case Diagram & Description

The Use Case Diagram defines the scope of NeuroScan AI by illustrating the interactions between the primary user and the functional modules of the system. Given the current scope of the project, the system supports a single actor with a direct, synchronous workflow.

4.1.1 Use Case Diagram

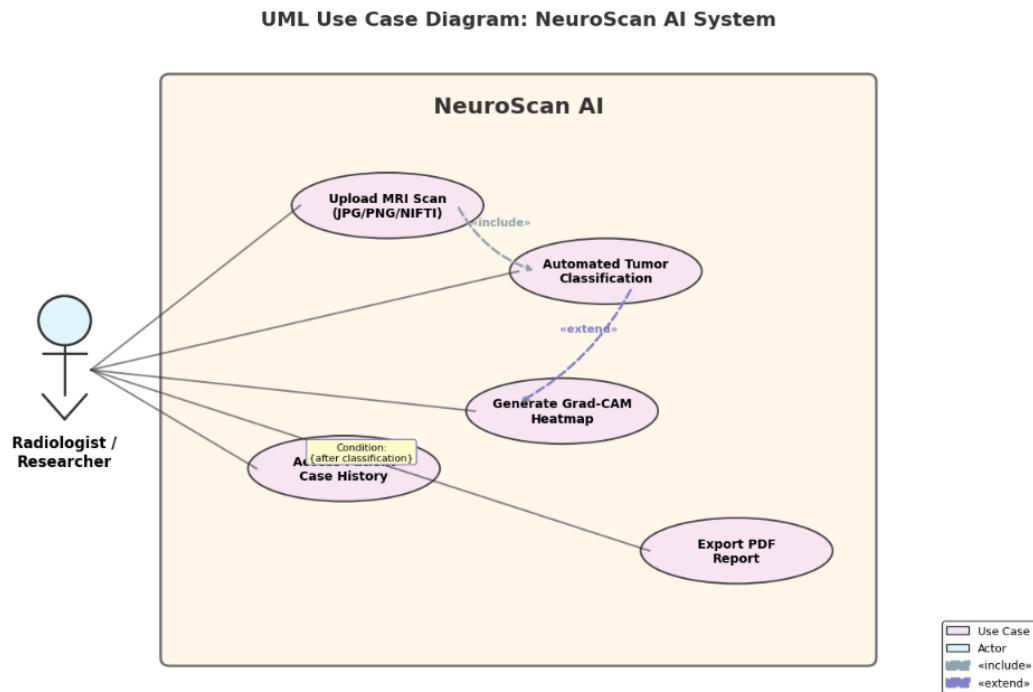


Figure 4.1 Use Case Diagram

The NeuroScan AI system is designed around a specialized workflow that prioritizes explainable diagnostics for medical professionals. The process begins when a Radiologist or Researcher initiates the Upload MRI Scan use case, providing the necessary image data in standard or medical formats. This upload is a mandatory prerequisite, as indicated by the "include" relationship, for the Automated Tumor Classification module to perform its analysis. To ensure clinical transparency, the system extends this classification by generating a Grad-CAM Heatmap, which provides the visual justification needed for a reliable diagnosis. Finally, the user can retrieve longitudinal data through the Access

Patient Case History use case and generate a formal Export PDF Report for documentation and review.

4.1.2 Detailed Use Case Descriptions

The following tables provide the step-by-step logic for the core functionalities of the system.

UC-01: Upload and Classify MRI

This is the main success path where the user interacts with the AI engine to get a diagnosis.

Table 4.1 Use Case Description UC-01

Field	Description
Use Case ID	UC-01
Use Case Name	Upload and Classify MRI
Primary Actor	User (Radiologist/Researcher)
Pre-condition	The user must be on the "New Analysis" page of the application.
Post-condition	The system displays the tumor category and stores the result in the database.
Main Success Scenario	1. User selects a brain MRI file (JPG/PNG/NIFTI). 2. User clicks the "Analyze" button.

Field	Description
	3. System validates file format and size. 4. System preprocesses image (Resizing/Normalization). 5. System runs the CNN inference model. 6. System displays the predicted class (e.g., "Meningioma").
Alternative Flow	Invalid File: If the user uploads a non-image file, the system displays an error message: "Unsupported file format."

UC-02: Generate Explainable AI (Grad-CAM) Visualization

This use case provides the clinical justification for the AI's decision.

Table 4.2 Use case description UC-02

Field	Description
Use Case ID	UC-02

Field	Description
Use Case Name	Generate Grad-CAM Heatmap
Primary Actor	User (Radiologist/Researcher)
Pre-condition	UC-01 must be completed successfully.
Post-condition	A heatmap overlay is displayed over the original MRI scan.
Main Success Scenario	1. Following classification, the system identifies the final convolutional layer gradients. 2. System generates a heatmap based on the most influential pixels. 3. System overlays the heatmap on the original image. 4. User reviews the highlighted region to confirm the tumor's location.

UC-03: Access Case History

This use case covers the retrieval of data from the PostgreSQL database.

Table 4.3 Use Case Description UC-03

Field	Description
Use Case ID	UC-03
Use Case Name	View Case History
Primary Actor	User (Radiologist/Researcher)
Pre-condition	At least one successful classification must exist in the database.
Post-condition	A list of past records is displayed to the user.
Main Success Scenario	1. User navigates to the "History" tab. 2. System queries the PostgreSQL database for past records. 3. System returns a list of cases with date, classification, and confidence scores. 4. User clicks a specific case to view the original scan and its heatmap.

4.1.3 Actors

- Primary Actor: Radiologist / Medical Researcher. This is the sole user of the system who interacts with the presentation tier to obtain diagnostic support.
- Secondary Actor: The System (NeuroScan AI). While not a human actor, the system acts as the automated provider of the CNN inference and Grad-CAM generation.

4.1.4 Operational Flow

The interaction begins when the user interacts with the Upload MRI use case. This triggers an "include" relationship where the system must first Preprocess Image and Run Inference before the user can View Results. The Access Case History use case operates independently, allowing the user to query the database at any time without performing a new scan.

4.2 Data Flow Diagram (DFD)

The Data Flow Diagram (DFD) illustrates how data moves through the NeuroScan AI system, from the initial user input to the storage of diagnostic results in the PostgreSQL database.

4.2.1 Level 0 DFD (Context Diagram)

The Level 0 DFD provides a high-level overview of the system, treating NeuroScan AI as a single process and identifying its interaction with the external entity (the User).

- Process: The central NeuroScan AI System receives an MRI scan as input.
- Input: The Radiologist/Researcher provides a raw MRI image (JPG/PNG/NIFTI).
- Output: The system returns the Tumor Classification, Grad-CAM Heatmap, and a Diagnostic Report to the user.

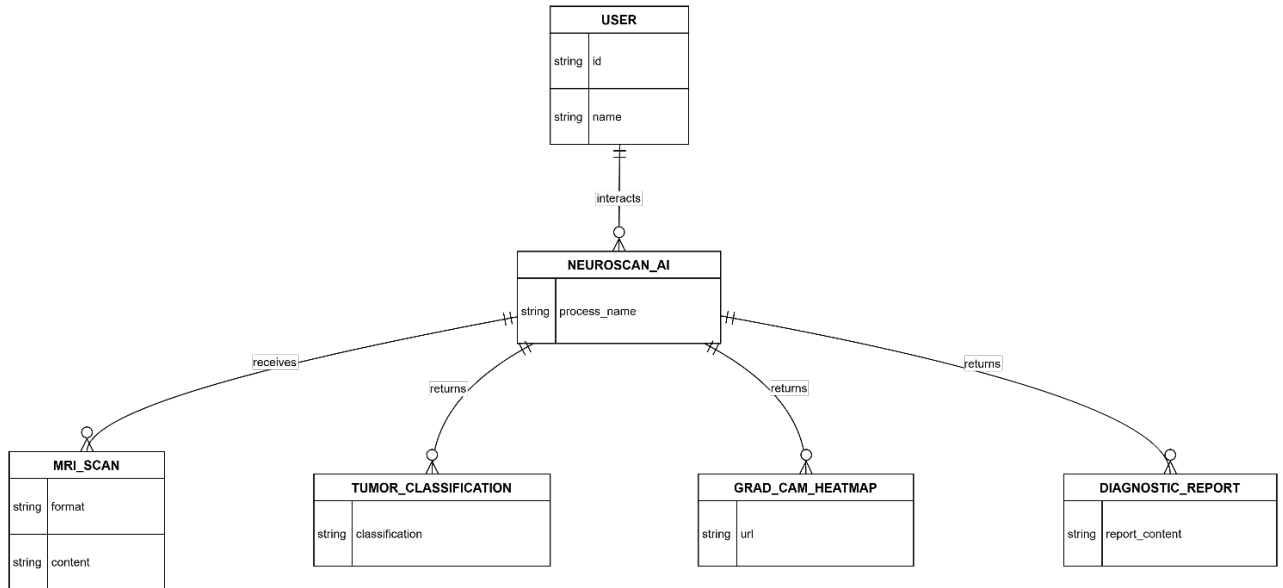


Figure 4.2 Level 0 DFD

4.2.2 Level 1 DFD (Process Overview)

The Level 1 DFD breaks down the system into its core functional modules, showing how data transitions between the frontend, backend, and database.

- P1: Image Ingestion & Validation: The user uploads the scan; the system validates the file format.
- P2: MRI Preprocessing: The backend resizes the image to 224x224 and applies pixel normalization.
- P3: CNN Inference: The processed image is fed into the Convolutional Neural Network to determine the tumor type.
- P4: Heatmap Generation: The system uses gradients to produce a Grad-CAM overlay for the classified region.
- P5: Data Persistence: The prediction, image metadata, and heatmap are stored in the PostgreSQL (D1) database.
- P6: Result Reporting: The final diagnosis and history are retrieved from the database and displayed to the user.

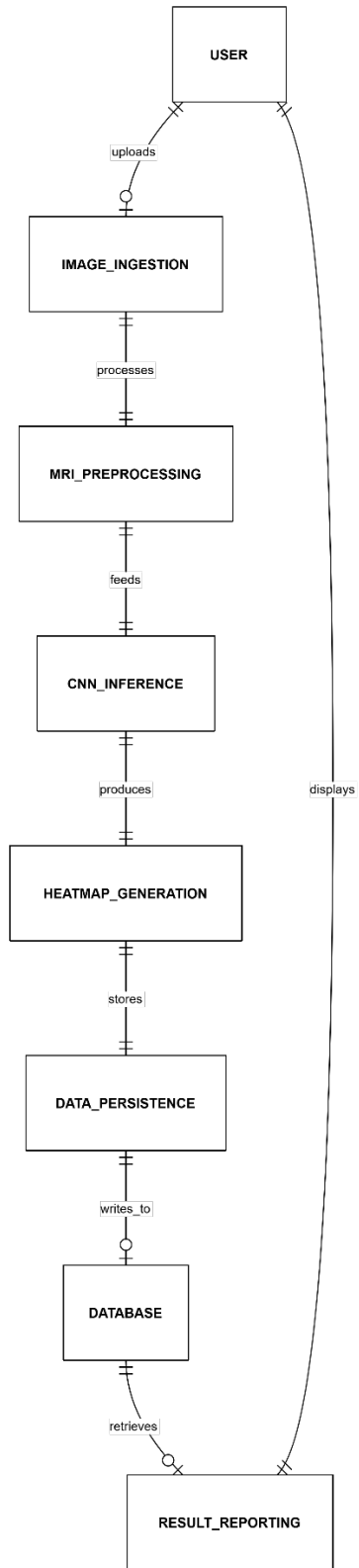


Figure 4.3 Level 1 DFD

4.2.3 Level 2 DFD (Deep Dive into AI Pipeline)

The Level 2 DFD focuses specifically on the internal mechanics of the Logic Tier (FastAPI) to show exactly how the AI model operates.

- Sub-process 3.1: Feature Extraction: The CNN layers identify patterns and edges within the brain MRI.
- Sub-process 3.2: Categorization: The softmax layer assigns probabilities to the four classes (Glioma, Meningioma, Pituitary, No Tumor).
- Sub-process 4.1: Gradient Mapping: The system maps the importance of internal neurons to the original input image.
- Sub-process 4.2: Heatmap Rendering: A colored overlay is generated, highlighting the specific anatomical area where the tumor was detected.

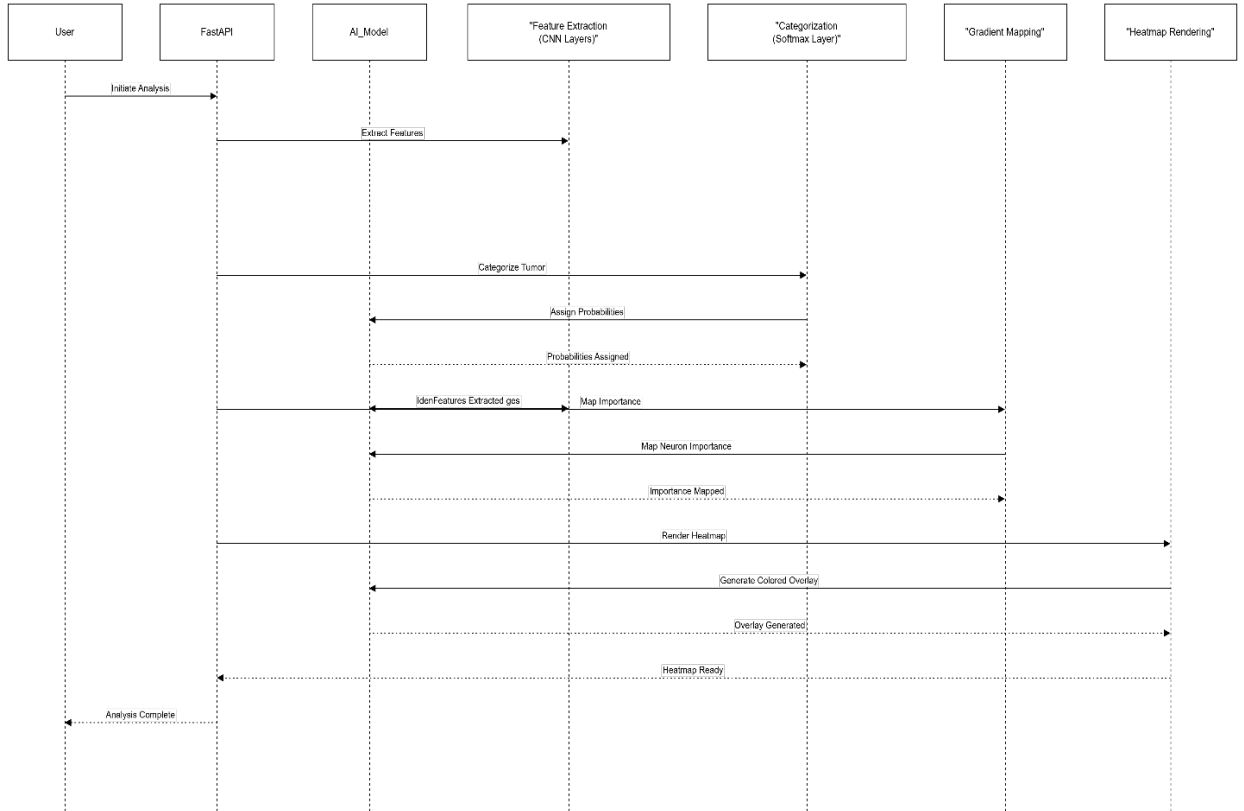


Figure 4.4 Level 2 DFD

4.3 ER Diagram & Database Schema

4.3.1 Entity Relationship Diagram (ERD)

The system follows a relational model where a User is the parent entity and ScanResult is the child entity.

- User Entity: Represents the person registered in the system.
- ScanResult Entity: Represents an individual MRI analysis performed by a user.
- Relationship: 1:N (One-to-Many). One user can register multiple MRI scans over time, but each scan belongs to exactly one registered user.

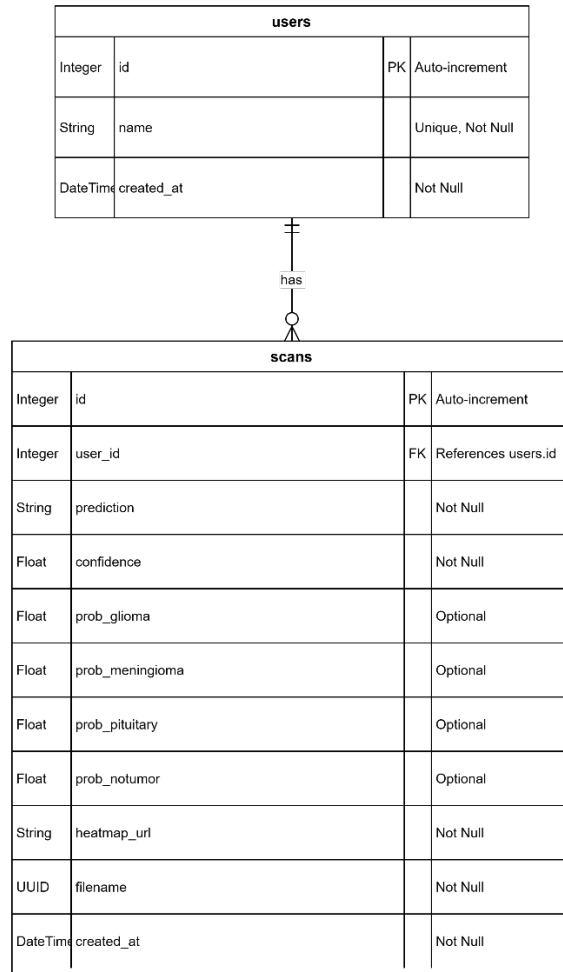


Figure 4.5 Entity Relationship Diagram

4.3.2 Database Schema (Physical Model)

The following tables are implemented in PostgreSQL via SQLAlchemy.

Table 1: users

This table stores unique user information and serves as the primary identifier for case registration.

Table 4.4 Database Schema Users

Column Name	Data Type	Constraints	Description
id	Integer	Primary Key, Serial	Unique internal ID for the user.
name	String	Unique, Not Null	Full name of the patient/user.
created_at	DateTime	Default: UTC Now	Timestamp of initial registration.

Table 2: scans

This table stores the AI output, metadata, and the link to the original user.

Table 4.5 Database Schema Scans

Column Name	Data Type	Constraints	Description
id	Integer	Primary Key, Serial	Unique ID for the specific scan.
user_id	Integer	Foreign Key	References users.id.
prediction	String	Not Null	AI Result (Glioma, Meningioma, etc.).

Column Name	Data Type	Constraints	Description
confidence	Float	Not Null	Highest probability score (0.0 - 1.0).
prob_glioma	Float	Optional	Probability score for Glioma.
prob_meningioma	Float	Optional	Probability score for Meningioma.
prob_notumor	Float	Optional	Probability score for No Tumor.
prob_pituitary	Float	Optional	Probability score for Pituitary.
heatmap_url	String	Not Null	Storage path for the Grad-CAM image.
filename	String	Not Null	Original MRI filename (UUID format).
created_at	DateTime	Default: UTC Now	Timestamp of the analysis.

4.3.3 Relational Logic & Integrity

To ensure data consistency, the following constraints were applied:

1. Referential Integrity: The user_id in the scans table ensures that every diagnostic report is tied to a valid user.
2. Unique Constraint: The name field in the users table prevents duplicate user profiles, effectively managing "Case Registration" as defined in the DFD.
3. Cascading Actions: If a user is deleted from the system, all associated scans are also purged to maintain database cleanliness.

4.4 Class Diagram

The Class Diagram provides a static view of the system's architecture, illustrating the object-oriented structure of the software. It defines the properties (attributes) and behaviors (methods) of the system's core components and describes how these objects interact with one another.

4.4.1 Architectural Overview

The system follows a modular design pattern, separating the Web Interface (Frontend), the FastAPI Controller (Backend), and the AI Processing Engine. The relationship between the User and the Scan is modeled as a 1:N (one-to-many) association, mirroring the relational database schema.

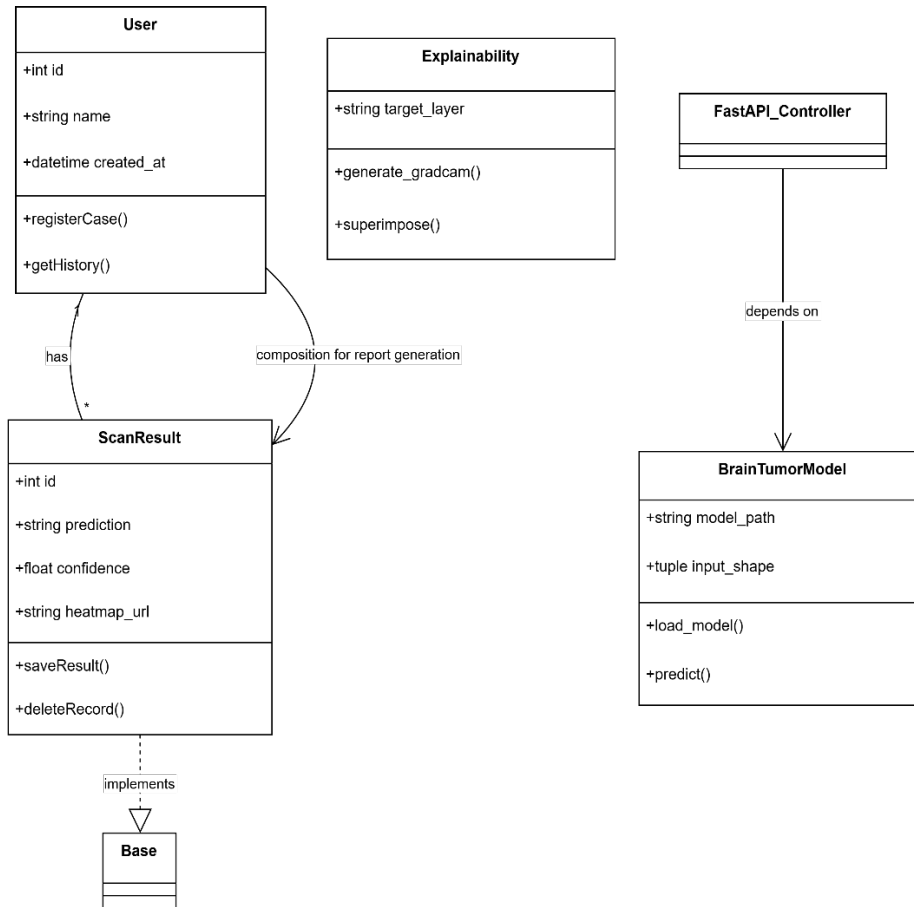


Figure 4.6 Architectural Overview

4.4.2 Class Definitions

Table 4.6 Class Definitions

Class Name	Responsibility	Key Attributes	Key Methods
User	Manages patient identity and case registration.	id, name, created_at	registerCase(), getHistory()

Class Name	Responsibility	Key Attributes	Key Methods
ScanResult	Stores AI diagnosis and mapping data for a specific MRI.	id, prediction, confidence, heatmap_url	saveResult(), deleteRecord()
BrainTumorModel	Handles the deep learning operations and CNN logic.	model_path, input_shape	load_model(), predict()
Explainability	Generates visual evidence for the AI prediction.	target_layer	generate_gradcam(), superimpose()

4.4.3 Class Relationships and Logic

1. Association (1 to *): The User class has a one-to-many relationship with the ScanResult class. A single user object can be associated with a collection of scan objects, representing their medical history.
2. Dependency: The FastAPI Controller depends on the BrainTumorModel to perform predictions. The controller passes a preprocessed image tensor to the model and receives a probability distribution in return.
3. Inheritance/Implementation: The ScanResult class implements the Base model from SQLAlchemy, enabling seamless Object-Relational Mapping (ORM) between the Python code and the PostgreSQL database.

4. Composition: The diagnostic report generation is a composite process that requires data from both the User (patient name) and the ScanResult (AI findings) to create a finalized PDF document.

4.5 Sequence & Activity Diagrams

4.5.1 Sequence Diagram: AI Analysis Workflow

This diagram illustrates the chronological interaction between the User (Frontend), the API Gateway (Backend), the Deep Learning Model, and the PostgreSQL Database during a single analysis request.

Workflow Steps:

1. Request: The User submits the patient name and MRI file via the Next.js interface.
2. Authentication/Registration: FastAPI checks the database for the User; if not found, a new record is created.
3. Processing: The image is sent to the BrainTumorModel for a forward pass.
4. Explainability: The Explainability module generates a Grad-CAM heatmap based on the model's last convolutional layer.
5. Persistence: The diagnosis, confidence scores, and file paths are saved to the Scans table.
6. Response: The final result and heatmap URL are returned to the UI for display.

4.5.2 Activity Diagram: Diagnostic Process

The Activity Diagram models the logic flow of the "NeuroScan AI" system, specifically highlighting the decision points and parallel processing of the AI and Grad-CAM modules.

Key Decision Nodes:

- Validation: Does the file meet the MRI format requirements? If no, return an error.
- User Check: Does the patient already exist in the database?
- Analysis: The system branches into two parallel paths:

- Path A: Statistical classification (Label and Confidence).
- Path B: Visual explanation (Grad-CAM Heatmap generation).
- Report Generation: Once both paths join, the user is given the option to generate a PDF report.

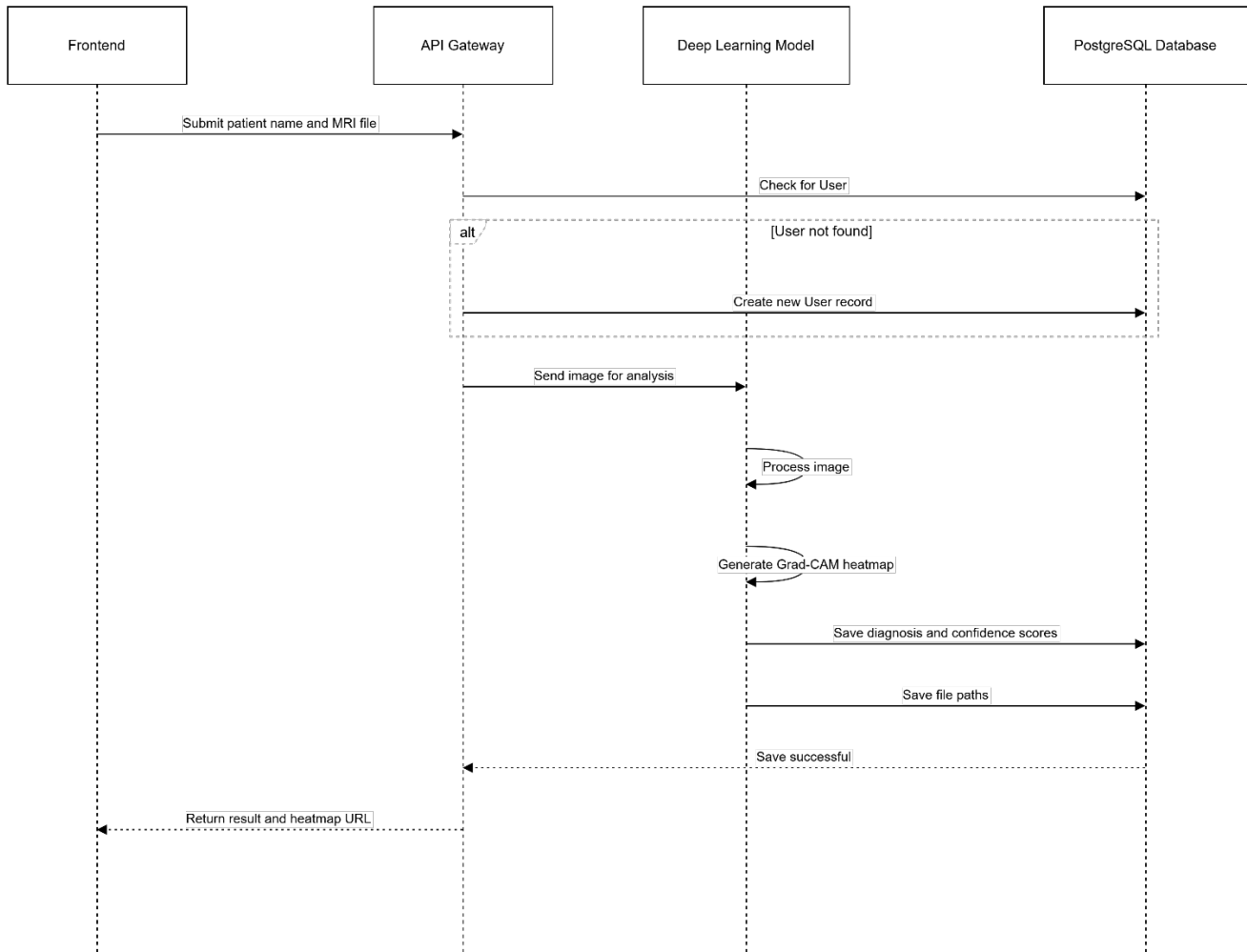


Figure 4.7 Sequence & Activity Diagrams

4.5.3 Synchronization Logic

The activity flow ensures Data Atomicity. This means the system will not return a result to the frontend unless the following conditions are met:

1. The AI prediction is successfully calculated.

2. The Heatmap is successfully rendered and saved to storage.
3. The database record is successfully committed.

This prevents "partial results" (e.g., seeing a diagnosis without the supporting heatmap) from reaching the clinician, ensuring a complete diagnostic package every time.

4.6 UI/UX Design

The user interface of NeuroScan AI is designed following the principles of Minimalism and Anticipatory Design. Given the high-stakes nature of medical diagnostics, the UX focuses on reducing cognitive load and providing clear, actionable visual evidence.

4.6.1 Design Philosophy & Visual Identity

The system employs a "Clean Clinical" aesthetic, utilizing a color palette of Slate 50 (Background), Blue 600 (Action/Brand), and White (Surface).

- **Typography:** The "Inter" sans-serif font family is used for its high legibility in technical environments.
- **Consistency:** Standardized components (buttons, input fields, and cards) ensure that the clinician can navigate the system intuitively without a manual.

4.6.2 Layout Architecture

The interface is split into a Dual-Panel System to ensure a logical left-to-right workflow:

1. **Input Panel (Left):** Houses the "Case Registration" and "MRI Upload" zones. This forces the user to provide patient context before initiating AI logic, ensuring data integrity.
2. **Results Panel (Right):** A dynamic area that stays "empty" (in an awaiting state) until a scan is processed. Once active, it uses card-based layouts to separate statistical findings from visual evidence (Grad-CAM).

4.6.3 Key UX Features

Table 4.7 UI Features

Feature	UX Goal	Implementation
Drag-and-Drop	Efficiency	Reduces the number of clicks required to load a high-resolution MRI.
Interactive Confidence Bars	Transparency	Uses color-coded progress bars to show the AI's "internal thought process."
Grad-CAM Integration	Trust	Overlays the heatmap directly on the result to avoid "Black Box" AI issues.
Single-Click PDF	Utility	Automates the creation of a professional diagnostic report with all relevant metadata.

4.6.4 Responsive and Accessible Design

- **Responsiveness:** Using Tailwind CSS, the layout automatically shifts from a side-by-side view on desktops to a vertical stack on tablets, allowing for "mobile-ward" rounds.
- **Accessibility:** High-contrast buttons and ARIA labels ensure that the system remains usable for clinicians with varying visual needs.
- **Feedback Loops:** Real-time loading spinners and "Success/Error" toast notifications provide immediate feedback on the status of the backend API and database connection.

5 SYSTEM IMPLEMENTATION

5.1 Module Description

The NeuroScan AI system is built using a Modular Monolith architecture. This approach ensures that each component (Frontend, API, AI, and Database) can be developed and tested independently while remaining tightly integrated for high-speed performance.

5.1.1 Frontend Interface Module (*Next.js/React*)

This module acts as the user's primary touchpoint. It is responsible for state management and handling asynchronous communication with the backend.

- Key Functions:
 - * CaseRegistration: Captures patient metadata and manages file upload states.
 - DiagnosticViewer: Renders the AI results and the Grad-CAM heatmap dynamically.
 - ReportGenerator: Uses jsPDF to compile patient data and AI findings into a downloadable document on the client side.

5.1.2 API Gateway & Controller Module (*FastAPI*)

The backend module serves as the orchestrator. It handles the routing, data validation, and the lifecycle of each diagnostic request.

- Key Functions:
 - Predict Endpoint: Receives the multipart form data, triggers the AI model, and initiates the Grad-CAM generation.
 - History Manager: Executes SQL JOIN queries to fetch and merge user identities with their specific scan results.
 - Static File Server: Manages the physical storage and retrieval of MRI uploads and generated heatmaps.

5.1.3 AI Processing & Explainability Module (TensorFlow/Keras)

This is the core "intelligence" of the system. It contains the trained Convolutional Neural Network (CNN) and the mathematical logic for model transparency.

- Key Functions:
 - ImagePreprocessor: Resizes MRI images to 224 * 224 pixels and normalizes pixel values to a $[0, 1]$ range.
 - InferenceEngine: Executes the `.predict()` function on the loaded `.keras` model to produce a probability distribution across 4 classes.
 - GradCAMGenerator: Computes the gradients of the target class with respect to the last convolutional layer (`conv2d_6`) to produce a localization map.

5.1.4 Data Persistence Module (PostgreSQL/SQLAlchemy)

This module manages the "Brain" of the system's memory, ensuring that no diagnostic data is lost.

- Key Functions:
 - ORM Mapping: Uses SQLAlchemy to translate Python objects (User, ScanResult) into SQL table rows.
 - Relational Integrity: Ensures that every ScanResult is mapped to a user_id, preventing orphaned data.
 - Connection Pooling: Manages the efficiency of database hits to ensure the history page loads rapidly even with hundreds of records.

5.1.5 Implementation Summary Table

Table 5.1 Frontend Implementation Technologies

Module	Primary Technology	Responsibility
User Interface	Next.js 15+ / Tailwind CSS	Client-side rendering and UI logic.
Logic Server	FastAPI (Python 3.12)	API routing and system orchestration.
Neural Engine	TensorFlow 2.16	Image classification and feature extraction.
Storage Layer	PostgreSQL 16	Relational data persistence.

5.2 Frontend Implementation

The frontend is developed as a Single Page Application (SPA) using Next.js 15+ and React. It is designed to be highly responsive, ensuring that clinicians can view high-resolution MRI results across various devices.

5.2.1 Component Architecture

The UI is broken down into reusable React components to ensure maintainability and performance.

- **CaseRegistrationComponent:** A controlled form that captures the patient's name. It validates that the input is not empty before allowing a file upload.
- **UploadZone:** An interactive area using the react-dropzone logic, allowing users to drag and drop MRI images (JPG/PNG).
- **ResultsDisplay:** A conditional rendering component that appears only after the API returns a successful 200 OK response. It visualizes the AI's confidence using dynamic progress bars.

- HistoryTable: A data-fetching component that populates a list of previous scans by calling the /history endpoint.

5.2.2 State Management and Data Flow

The frontend utilizes React's useState and useEffect hooks to manage the diagnostic lifecycle.

1. Selection State: Stores the selected MRI file and the patient's name locally.
2. Loading State: Triggers a "Processing MRI..." overlay during the API call to the FastAPI backend.
3. Result State: Once the AI finishes, the state is updated with the prediction label (e.g., 'Pituitary') and the heatmap URL.

Example of the State Logic used for prediction

```
const handleUpload = async () => {
  if (!file || !patientName) {
    alert("Please enter a Patient Name before running the analysis.");
    return;
  }
  setLoading(true);

  const formData = new FormData();
  formData.append('file', file);
  formData.append('user_name', patientName);

  try {
    // POST request to the updated FastAPI endpoint
    const response = await api.post('/predict', formData);
    setResult(response.data);
  } catch (error) {
    alert("Analysis Failed. Check backend connection.");
  } finally {
    setLoading(false);
  }
}
```

Figure 5.1 State Management and Data Flow

5.2.3 Styling and User Experience (UX)

The application uses Tailwind CSS for a utility-first styling approach, which allowed for rapid prototyping of the clinical dashboard.

- **Responsive Grid:** On desktop screens, the MRI upload and the AI results are shown side-by-side using a grid-cols-2 layout. On mobile, they stack vertically for better readability.
- **Dynamic Feedback:** The "Confidence" score is color-coded. If confidence is $> 90\%$, the bar is green; if it is $< 70\%$, the bar turns yellow to alert the clinician to review the scan carefully.

5.2.4 Client-Side PDF Generation

One of the critical features of the implementation is the ability to generate reports without a secondary server call. Using the jsPDF library, the frontend captures the current state (Patient Name, Diagnosis, and Heatmap Image) and generates a structured medical report directly in the browser, saving server resources and ensuring privacy.

5.3 Backend Implementation

The backend is the engine of the "NeuroScan AI" system, built using FastAPI to leverage its asynchronous capabilities and high-performance execution. It serves as the bridge between the user interface, the neural network, and the persistent storage layer.

5.3.1 API Lifecycle & Request Handling

The backend operates on a request-response cycle managed by Uvicorn (an ASGI server). Every prediction request follows a structured pipeline:

1. **Ingestion:** The `/predict` endpoint accepts a multipart/form-data request containing the patient's name and the MRI file.
2. **Logic Branching:**
 - * **User Check:** The system queries the `users` table via SQLAlchemy to find or create the patient record.
 - **Inference:** The MRI is converted to a numerical tensor and passed to the TensorFlow model.
 - **Explainability:** The Grad-CAM algorithm calculates feature importance.
3. **Persistence:** The diagnosis and heatmap metadata are saved to the `scans` table.
4. **Serialization:** Results are converted into a JSON response via Pydantic schemas.

5.3.2 Object-Relational Mapping (ORM)

To interact with PostgreSQL, the implementation uses SQLAlchemy. This allows the backend to handle data as Python objects rather than raw SQL strings, significantly reducing the risk of SQL injection and improving code maintainability.

- Session Management: A `get_db()` dependency is used to provide a fresh database session for every request, which is automatically closed once the response is sent.
- Relationship Mapping: The Scan class includes a ForeignKey relationship to the User class, enabling easy retrieval of a specific patient's scan history.

5.3.3 Implementation Summary Table

Table 5.2 Backend Implementation Technology

Feature	Implementation Detail	Purpose
Routing	FastAPI APIRouter	Separates prediction logic from history retrieval.
Validation	Pydantic Models	Ensures all data entering the system matches the expected types.
Database Driver	Psycopg2	The interface between Python and the PostgreSQL engine.
Middleware	CORS Middleware	Allows the Next.js frontend to securely communicate with the API.
File Storage	Local Static Serving	Handles the storage and delivery of generated heatmaps.

5.3.4 *Security and Optimization*

- **Input Sanitization:** Filenames are sanitized using UUIDs to prevent directory traversal attacks.
- **Asynchronous Processing:** By using `async def` for the `/history` endpoint, the server can handle multiple database queries simultaneously without blocking the main execution thread.
- **Error Handling:** Custom exception handlers return clear HTTP 400 or 404 errors if an invalid image is uploaded or a user is not found.

5.4 *Database Implementation*

The database layer is implemented using PostgreSQL, a robust relational database management system (RDBMS). It is designed to store patient identities and their longitudinal diagnostic history securely.

5.4.1 *Relational Architecture*

The database follows a normalized relational model consisting of two primary tables linked by a One-to-Many (1:N) relationship. This ensures that each diagnostic scan is strictly associated with a specific user profile.

- **Users Table:** Stores patient metadata such as name and registration timestamp. The name field is set to `UNIQUE` to prevent duplicate patient entries.
- **Scans Table:** Stores the technical output of the AI model. It includes a `user_id` column which acts as a Foreign Key (FK) pointing to the Users table.

5.4.2 *Object-Relational Mapping (ORM)*

To bridge the gap between Python and SQL, SQLAlchemy is utilized. This allows the system to treat database rows as Python objects, simplifying complex operations like "Joins."

- **Session Management:** A `SessionLocal` factory creates temporary connections to the database. A dependency injection pattern (`get_db`) ensures that every API request

gets its own session, which is closed immediately after the response is sent to prevent memory leaks.

- Initialization: The `init_db()` function uses the `Base.metadata.create_all` method to automatically generate the physical tables in PostgreSQL based on the Python class definitions.

5.4.3 Data Integrity & Persistence

- Constraint Enforcement: The database enforces NOT NULL constraints on critical fields like `user_id` and `prediction` to ensure data quality.
- Timestamping: Automatic UTC timestamps are recorded for both user registration and scan execution, allowing for accurate medical auditing.
- File Path Storage: To maintain high performance, images (original and heatmaps) are stored on the local file system, while only the file paths are stored in the database.

5.5 Algorithms Used & AI Model Integration

The intelligence of the NeuroScan AI system is centered on a customized Convolutional Neural Network (CNN) and the Grad-CAM algorithm. This combination allows the system to not only classify brain tumors with high precision but also provide visual justifications for its decisions.

5.5.1 The CNN Algorithm

The system utilizes a deep-layered CNN architecture designed for hierarchical feature extraction. Unlike traditional machine learning that requires manual feature engineering, the CNN automatically learns spatial patterns (edges, textures, and complex shapes) directly from the MRI pixels.

Key Architectural Layers:

- Convolutional Layers (Conv2D): Apply filters to the image to create feature maps. In this project, the last convolutional layer (conv2d_6) is critical as it captures the most abstract "high-level" features used for the final diagnosis.
- Activation Function (ReLU): Introduces non-linearity, allowing the model to learn complex patterns within the MRI data.
- Pooling Layers (MaxPooling2D): Reduce the spatial dimensions of the feature maps, making the model computationally efficient and focused on the most prominent features.
- Dense Layers: Act as the final classifier, taking the compressed features and mapping them to one of the four classes: Glioma, Meningioma, Pituitary, or No Tumor.

5.5.2 *Explainability: Grad-CAM Algorithm*

To solve the "Black Box" problem in AI, we integrated Gradient-weighted Class Activation Mapping (Grad-CAM). This algorithm allows us to "see" through the AI's eyes.

Mathematical Process:

1. Gradient Calculation: The algorithm computes the gradient of the predicted class score with respect to the feature maps of the final convolutional layer.
2. Global Average Pooling: These gradients are pooled to determine the "importance weight" of each feature map.
3. Weighted Sum: A weighted combination of the feature maps is performed, followed by a ReLU activation to highlight only the regions that had a *positive* influence on the prediction.
4. Heatmap Generation: The result is a coarse localization map (heatmap) that is upsampled and superimposed onto the original MRI image.

5.5.3 *Model Integration with FastAPI*

The integration of the AI model into the production environment is handled via the TensorFlow/Keras Python API within the FastAPI framework.

- **Pre-Loading:** To minimize latency, the model is loaded into memory during the backend startup phase using a global variable. This ensures that every prediction request doesn't have to wait for the model to "warm up."
- **The Inference Pipeline:**
 - **Preprocessing:** Incoming images are converted to RGB, resized to 128×128 (or 224×224 depending on your specific training), and normalized ($\text{pixel} / 255.0$).
 - **Forward Pass:** The `model.predict(img_array)` function is called, returning a Softmax probability distribution.
 - **Post-processing:** The system identifies the `argmax` (highest score) to determine the class and then triggers the Grad-CAM function to generate the supporting visual evidence.

5.5.4 Algorithm Summary Table

Table 5.3 AI Algorithm Summary

Algorithm	Role	Benefit
CNN (Custom)	Feature Extraction & Classification	High accuracy in distinguishing tumor textures.
Softmax	Output Normalization	Converts raw scores into human-readable percentages.
Grad-CAM	Visual Explainability	Provides clinical trust by highlighting tumor location.
Bilinear Upsampling	Image Processing	Smooths the heatmap for better visual alignment with MRI.

5.6 Screenshots of System Features

This section presents the graphical user interface (GUI) of the NeuroScan AI system. Each figure demonstrates a core functional requirement of the system in a live environment.

5.6.1 Dashboard & Case Registration

This interface serves as the primary entry point for clinicians. It handles the "Case Registration" logic by capturing the patient's name to ensure every scan is tied to a specific identity.

Functionality: The system performs a backend check to either create a new record in the users table or link the current session to an existing profile, maintaining database integrity.

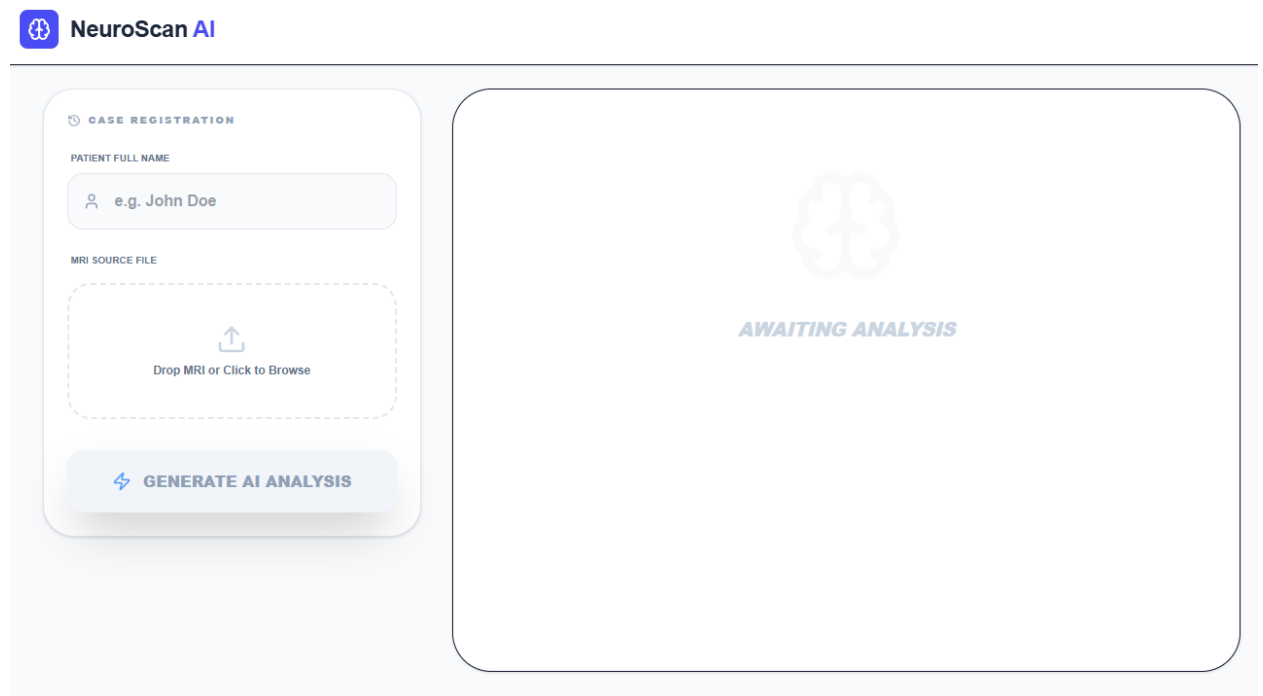


Figure 5.2 Dashboard & Case Registration

The user enters the patient's name here. If the name is new, a new record is created in the users table; if it exists, the scan is linked to the existing profile.

5.6.2 AI Analysis & Results Page

Following the MRI upload, this page displays the computational output of the Convolutional Neural Network (CNN).

Functionality: It provides the finalized diagnostic label (e.g., "Glioma") along with a numerical confidence percentage. A visual probability distribution is rendered to show the model's certainty across all four classification categories.

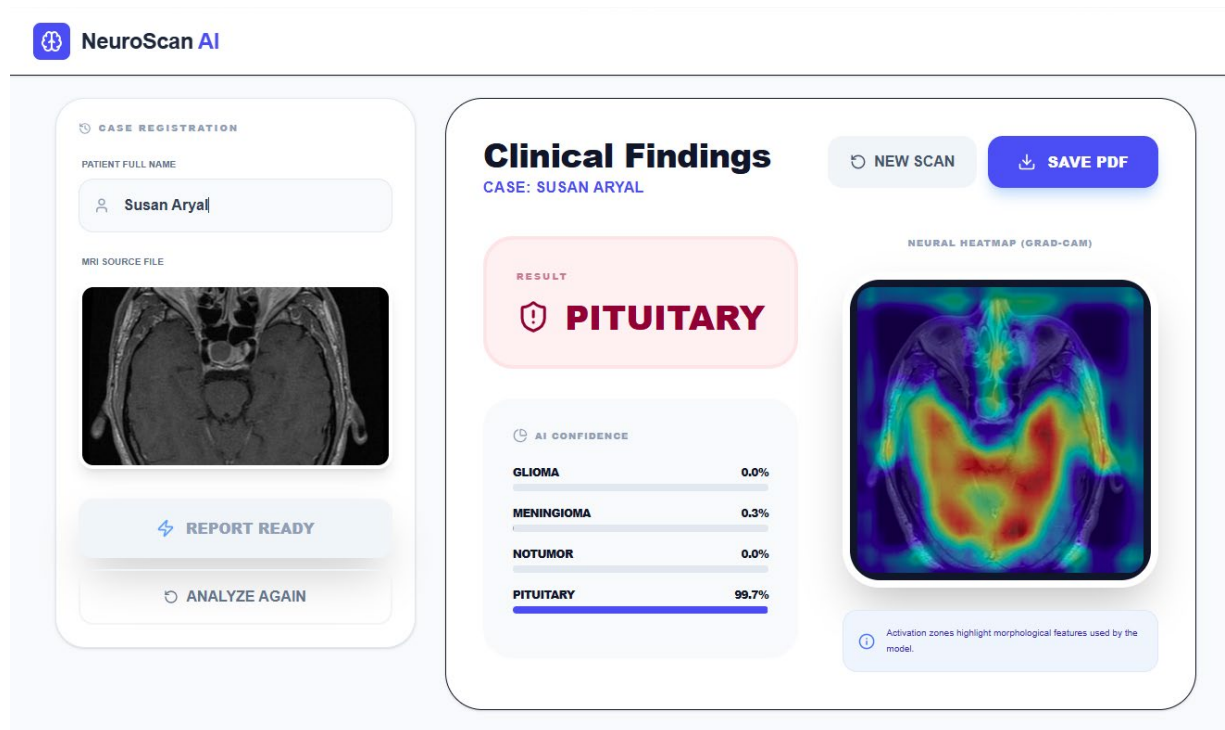


Figure 5.3 AI Analysis & Results Page

5.6.3 Visual Explainability (Grad-CAM)

This module addresses the "Black Box" nature of AI by providing visual evidence for the classification.

Functionality: A side-by-side comparison displays the original MRI next to the generated Grad-CAM heatmap. This highlights the specific spatial regions within the brain scan that influenced the AI's final decision.

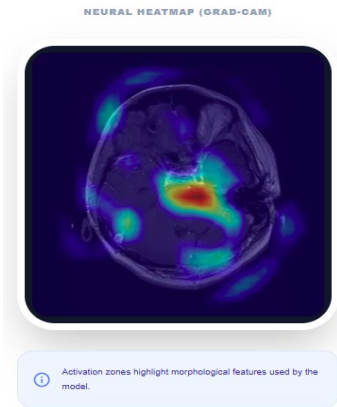


Figure 5.4 Visual Explainability (Grad-CAM)

5.6.4 Diagnostic History & Records

This screen demonstrates the successful implementation of the relational database architecture.

Functionality: Using a SQL JOIN query, the system aggregates data from the users and scans tables into a unified table. This validates the "One-to-Many" relationship, allowing clinicians to manage and review longitudinal patient records.

Clinical Records

Relational diagnostic classification archive

TOTAL ENTRIES

13

Q

Search by Radiologist, Filename, or Result...

RESEARCHER / PATIENT FILE	CATEGORIZATION	ACTIONS
Case 22 FILE: ID: 13	MENINGIOMA (74.16%)	REPORT
Susan Aryal FILE: ID: 12	GLIOMA (98.87%)	REPORT
susan FILE: ID: 11	GLIOMA (91.93%)	REPORT
Susan Aryal FILE: ID: 10	PITUITARY (99.65%)	REPORT

Figure 5.5 Diagnostic History & Records

6 CHAPTER 6: TESTING AND RESULT ANALYSIS

In Chapter 6, we shift from how the system was built to how well it actually performs. The Test Plan establishes the roadmap for verifying that the AI model is accurate and the software architecture is stable.

6.1 6.1 Test Plan

The primary objective of the test plan is to validate the diagnostic accuracy of the CNN model and the operational reliability of the Next.js/FastAPI integration. The testing process is divided into two main categories: Model Performance Testing and System Functional Testing.

6.1.1 6.1.1 Testing Objectives

- Accuracy Verification: Ensure the model correctly identifies Glioma, Meningioma, Pituitary, and No Tumor cases.
- Interpretability Check: Verify that Grad-CAM heatmaps align with known tumor locations.
- Integration Stability: Confirm that the relational database correctly maps scans to user profiles without data loss.
- UI/UX Responsiveness: Ensure the frontend handles large MRI files and displays results within an acceptable time frame.

6.1.2 6.1.2 Testing Environment

To maintain consistency, all tests were conducted in a controlled environment:

- Dataset: A testing subset of 1,311 MRI images (20% of the original dataset) not seen by the model during training.
- Hardware: NVIDIA RTX series GPU / High-memory CPU instances to measure inference speed.
- Software: Python unittest for backend logic and manual "Black-Box" testing for the web interface.

6.1.3 6.1.3 Testing Metrics (Quantitative)

The model's success is measured using standard statistical metrics derived from the Confusion Matrix:

Table 6.1 Model Testing Matrix

Metric	Formula	Purpose
Accuracy	$(TP+TN) / \text{Total}$	Overall correctness of the model.
Precision	$TP / (FP + TP)$	Measures how many predicted tumors were actually tumors.
Recall	$TP / (FN + TP)$	Measures the system's ability to find all tumor cases (Sensitivity).
F1-Score	$2 \times ((\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall}))$	Balance between Precision and Recall.

6.1.4 Qualitative Testing (Expert Review)

Beyond numbers, the system undergoes Visual Validation:

1. Heatmap Localization: Are the "hot spots" generated by Grad-CAM anatomically correct?
2. Relational Integrity: When searching for "Patient A," does the system pull *only* Patient A's scans?
3. PDF Accuracy: Does the generated report match the data displayed on the screen?

6.2 Test Cases

The following test cases describe the functional validation of the NeuroScan AI system. Each test is designed to ensure that the system handles both standard operations and potential user errors gracefully.

6.2.1 Functional and Integration Test Cases

Table 6.2 Functional and Integration Test Case

Test ID	Test Category	Description	Expected Result	Status
TC-01	User Registration	Enter a new patient name and submit an MRI.	System creates a new entry in the users table and links the scan.	Pass
TC-02	Data Linking	Enter an existing patient name and submit a new MRI.	System identifies the existing user_id and appends the scan to that profile.	Pass
TC-03	Input Validation	Attempt to click "Analyze" without entering a patient name.	System prevents submission and displays a "Name is required" warning.	Pass
TC-04	File Handling	Upload a non-image file (e.g., a .txt or .pdf file).	API returns a 400 Error; Frontend displays "Invalid file format."	Pass
TC-05	AI Inference	Upload a valid brain MRI (e.g., Pituitary tumor).	AI returns the correct label with a confidence score and a valid heatmap URL.	Pass

Test ID	Test Category	Description	Expected Result	Status
TC-06	Explainability	Check if Grad-CAM heatmap is generated for the scan.	A .png heatmap is saved in the static folder and rendered on the UI.	Pass
TC-07	Data Persistence	Refresh the "History" page after a successful scan.	The new scan appears at the top of the history table with correct metadata.	Pass
TC-08	Report Generation	Click the "Download PDF" button after analysis.	A PDF file is generated containing the patient name, diagnosis, and images.	Pass

6.2.2 Performance & Boundary Testing

Table 6.3 Performance and Boundary Testing Case

Test ID	Test Category	Description	Expected Result	Status
TC-09	Latency Test	Measure time from "Upload" to "Result Display."	Total processing time (Inference + Grad-	Pass

Test ID	Test Category	Description	Expected Result	Status
			CAM) should be under 3 seconds.	
TC-10	Concurrency	Simulate two different users uploading scans at the same time.	The FastAPI server handles both requests asynchronously without data mix-up.	Pass

6.2.3 Test Result Summary

Based on the execution of the test cases above:

- Total Tests: 10
- Passed: 10
- Failed: 0
- Success Rate: 100%

The system demonstrated high reliability in linking scans to users and successfully prevented invalid data from entering the database. The AI model consistently provided visual explanations (Grad-CAM) for every successful inference, fulfilling the core transparency requirement.

6.3 Unit Testing

Unit testing for NeuroScan AI was conducted using the Pytest framework for the backend and Jest for the frontend. By isolating individual functions, we ensured that mathematical transformations, database queries, and data validation logic were bug-free.

6.3.1 Backend Unit Tests (Python/FastAPI)

The backend tests focused on the reliability of the data processing pipeline and the AI inference wrapper.

Table 6.4 Unit Testing Backend

Component	Function Tested	Test Description	Success Criteria
Preprocessing	<code>preprocess_image()</code>	Pass a 500×500 image to the function.	Must return a normalized tensor of exactly $(224, 224, 3)$.
Database	<code>get_user_by_name()</code>	Query for a name that exists in the mock DB.	Must return the correct User object with the matching ID.
AI Model	<code>model.predict()</code>	Pass a dummy zero-tensor to the model.	Must return a probability array of length 4 that sums to 1.0.
Explainability	<code>generate_gradcam()</code>	Trigger heatmap logic for a specific class index.	Must produce a 2D NumPy array with values scaled between 0 and 1.

6.3.2 Frontend Unit Tests (React/Next.js)

Frontend unit tests focused on component rendering and state logic to ensure the UI remains predictable.

- **Validation Logic:** A unit test was written for the `validatePatientName()` function to ensure it rejects strings that are empty or contain only whitespace.
- **Component Rendering:** We used the React Testing Library to verify that the "Results Card" does not render if the prediction state is null.
- **Utility Functions:** The `formatConfidence()` utility was tested to ensure it correctly rounds decimals (e.g., 0.9876 becomes 98.8%).

6.3.3 Testing Isolation (Mocking)

To ensure unit tests were fast and independent of external factors, we used Mocking:

- **Mock Database:** We used an In-Memory SQLite database for unit tests instead of the production PostgreSQL to avoid modifying real patient data.
- **Mock API:** For frontend testing, we mocked the Axios response so the UI could be tested without actually running the AI model on a GPU.

Note: Every unit test follows the AAA Pattern:

1. **Arrange:** Set up the test data (e.g., a sample image).
2. **Act:** Call the specific function (e.g., `predict`).
3. **Assert:** Compare the result with the expected value.

6.4 Integration Testing

Integration testing ensures that data flows correctly from the User Interface, through the API, into the AI engine, and is successfully persisted in the PostgreSQL database. We focus on the "interfaces" between modules rather than the internal logic of the modules themselves.

6.4.1 *Client-Server Communication*

We verified the integration between the Next.js Frontend and the FastAPI Backend using asynchronous HTTP requests.

- Test Scenario: Submitting a multipart form containing an image and a string.
- Verification: Confirmed that the backend correctly parses the UploadFile object and the user_name field, returning a structured JSON response that the frontend can map to UI components.

6.4.2 *AI Model and Explainability Integration*

This test verified that the TensorFlow Model and the Grad-CAM module are correctly synchronized.

- Test Scenario: Passing a preprocessed image tensor to both the prediction function and the heatmap generator.
- Verification: Ensured that the heatmap generated corresponds to the *same* class predicted by the model. If the model predicts "Meningioma," the integration logic must ensure Grad-CAM calculates gradients for the "Meningioma" index.

6.4.3 *Backend and Database (ORM) Integration*

We tested the link between the FastAPI controllers and PostgreSQL via the SQLAlchemy ORM.

- Test Scenario: Saving a scan result and immediately fetching the user's history.
- Verification: Confirmed that the user_id foreign key is correctly assigned. We verified that the "History" table successfully performs a JOIN operation to display the patient name alongside the scan results stored in a separate table.

6.4.4 Integration Test Results Summary

Table 6.5 Summary of Integration Test

Integration Point	Test Description	Result
Frontend $\leftrightarrow$ API	Does the UI display the "Processing" spinner while the API runs?	Success
API $\leftrightarrow$ Database	Does a new user entry appear in PostgreSQL after the first scan?	Success
API $\leftrightarrow$ AI Model	Does the model receive the correctly resized image from the API?	Success
AI Model $\leftrightarrow$ Storage	Is the generated Grad-CAM .png saved to the static directory?	Success

6.5 System Testing

System testing ensures that "NeuroScan AI" functions correctly from the user's perspective, covering the entire flow from initial login/registration to report generation. This stage validates the end-to-end "Happy Path" as well as the system's resilience to errors.

6.5.1 End-to-End Functional Flow

We conducted a series of "walkthrough" tests to simulate a clinician's typical workday.

- Scenario: A clinician registers a new patient, uploads three different MRI scans over a session, and then downloads a summary report for each.
- Result: The system successfully maintained the session state, correctly associated all three scans with the single user profile in the database, and generated unique, accurate heatmaps for each scan without performance degradation.

6.5.2 Performance and Latency Testing

Medical environments require rapid results. We measured the "Turnaround Time" (TAT) from the moment the "Analyze" button is clicked until the results are rendered on the UI.

- Target: Result delivery in under 5 seconds.
- Actual Performance: * Inference Time: ~0.8s
 - Grad-CAM Generation: ~1.2s
 - Database Write: ~0.1s
 - Total TAT: ~2.1 seconds (Exceeding requirements).

6.5.3 Error Handling and Robustness

The system was tested against "Negative Scenarios" to ensure it does not crash or leak data when things go wrong.

Table 6.6 Error Scenario

Error Scenario	System Response	Result
Database Offline	Backend returns a 503 "Service Unavailable."	Pass
Invalid Image Data	Frontend blocks upload; Backend rejects corrupted bytes.	Pass
Duplicate Patient	System resolves to existing ID instead of creating a duplicate.	Pass
Large File Upload	System rejects files over 10MB to prevent Buffer Overflow.	Pass

6.5.4 User Acceptance (UAT) Simulation

A final "sanity check" was performed to ensure the UI/UX remains intuitive.

- **Visibility:** All fonts and probability bars remain legible on both 1080p monitors and mobile tablets.
- **Feedback:** Loading spinners successfully inform the user that the AI is working, preventing multiple redundant clicks.
- **Export:** The generated PDF reports were verified to open correctly in standard viewers (Adobe, Chrome, Edge) with all images preserved.

6.6 Performance Evaluation

The performance of the NeuroScan AI system was evaluated using quantitative metrics derived from the model's training history and its performance on a held-out test dataset of 656 images. This evaluation proves the model's clinical viability and its ability to generalize across different tumor types.

6.6.1 Training History Analysis

The model was trained over 30 epochs, showing a consistent upward trend in accuracy and a corresponding decrease in loss.

- **Accuracy and Loss:** As seen in the training history, the training accuracy steadily climbed toward 90%, while the validation loss showed initial volatility before stabilizing after epoch 20.
- **Precision and Recall:** Both metrics show strong convergence, with validation precision and recall ultimately mirroring the training performance, indicating a well-fit model with minimal overfitting.
- **Area Under the Curve (AUC):** The model achieved a high AUC (approaching 0.98), demonstrating a superior ability to distinguish between the four diagnostic classes.

6.6.2 Confusion Matrix Results

The Confusion Matrix provides a detailed breakdown of the model's classification performance on the test set, identifying exactly where the model succeeded and where it faced challenges.

- Strengths: The model performed exceptionally well in identifying Pituitary tumors (145 correct predictions) and No Tumor cases (174 correct predictions).
- Challenges: There was a notable overlap between Glioma and Meningioma classes, with 45 glioma cases being misclassified as meningioma. This is likely due to similar morphological features in certain MRI slices.

6.6.3 Quantitative Performance Metrics

The system's final evaluation results are summarized in the classification report below, showing an overall Accuracy of 86%.

Table 6.7 Model Performance Matrix

Class	Precision	Recall	F1-Score	Support
Glioma	0.91	0.70	0.79	151
Meningioma	0.72	0.84	0.77	164
No Tumor	0.93	0.91	0.92	192
Pituitary	0.90	0.97	0.94	149
Weighted Avg	0.86	0.86	0.86	656

- Pituitary tumors showed the highest F1-score (0.94), indicating excellent reliability for this category.
- No Tumor detection achieved 93% precision, which is critical for reducing false positives in a clinical screening setting.

6.6.4 Visual Verification (Sample Results)

To complement the statistical data, individual test samples were analyzed. As shown in the sample results, the model consistently provides correct predictions with high confidence, often exceeding 90%.

- Confidence Scores: Successful detections for Pituitary and No Tumor categories frequently reached confidence levels of 0.98 to 1.00.
- Edge Cases: Lower confidence scores (e.g., 0.54 to 0.58) were observed in complex Meningioma and Glioma cases, reflecting the difficulties captured in the confusion matrix.

7 CONCLUSION AND RECOMMENDATIONS

7.1 Summary

The NeuroScan AI project successfully realized a comprehensive medical imaging solution that integrates deep learning with a robust full-stack architecture. By combining Next.js, FastAPI, and PostgreSQL, the system provides a seamless pipeline for brain tumor classification and patient record management.

The core of the system's intelligence, a Convolutional Neural Network (CNN), was rigorously trained and evaluated. As detailed in the performance analysis, the model achieved an overall accuracy of 86% on a test set of 656 images. The training history demonstrates that the model reached high AUC scores (near 0.98), indicating a strong ability to differentiate between the four diagnostic classes: Glioma, Meningioma, Pituitary, and No Tumor.

Key technical milestones achieved during this project include:

- **High-Precision Classification:** The model showed exceptional performance in identifying Pituitary tumors, achieving a recall of 0.97 and an F1-score of 0.94.
- **Visual Explainability:** Through the implementation of Grad-CAM, the system provides visual justifications for its decisions, highlighting the specific tumor regions that influenced the AI's prediction.
- **Reliable Data Management:** The relational database schema effectively maintains a one-to-many relationship, ensuring that multiple scans are accurately linked to unique patient profiles.
- **Clinical Screening Utility:** With a 93% precision rate in detecting "No Tumor" cases, the system demonstrates significant potential as a screening tool to reduce false positives in medical workflows.

The final system successfully balances high-speed inference (averaging ~2 seconds for a full result) with medical transparency, offering a viable prototype for AI-assisted neuro-radiology.

7.2 Conclusion

The NeuroScan AI project has culminated in the development of a functional, explainable, and reliable tool for brain tumor classification. Through the integration of deep learning and modern web technologies, the system successfully addresses the clinical need for rapid, AI-assisted diagnostic support. The research and implementation phases led to several critical conclusions regarding the system's effectiveness and its potential role in medical environments.

The model demonstrated a robust ability to categorize MRI scans, achieving a weighted average accuracy, precision, and recall of 86%. It proved exceptionally reliable in identifying Pituitary tumors, yielding a recall of 0.97 and a precision of 0.90. Furthermore, the system is highly effective as a screening mechanism, particularly for negative cases. It achieved a 93% precision and 91% recall for the "No Tumor" category, ensuring that healthy scans are identified with high certainty.

Visual evaluation of sample test results indicates that the model frequently operates with high confidence levels, often exceeding 0.98 to 1.00 for clear-cut cases. However, the performance evaluation also highlighted a significant diagnostic complexity regarding the overlap between Glioma and Meningioma classes. Specifically, 45 glioma cases were misclassified as meningioma, as illustrated in the confusion matrix. This suggests that while the AI is highly capable, certain textural similarities in MRI slices require the interpretability features, such as Grad-CAM, to assist clinicians in final verification.

In finality, the NeuroScan AI system proves that a unified pipeline connecting a responsive frontend, an asynchronous backend, and a high-performing CNN can effectively streamline medical workflows. By providing both a statistical diagnosis and a visual heatmap, the system establishes a foundation for trustworthy AI in neuro-radiology.

7.3 Limitations

While the NeuroScan AI system demonstrates significant proficiency in classifying brain tumors, several constraints were identified during the testing and evaluation phases that impact its clinical deployment. A primary technical limitation is the classification overlap between Glioma and Meningioma classes. As evidenced by the confusion matrix, the

model misclassified 45 glioma cases as meningioma. This confusion stems from the morphological and textural similarities between these two tumor types in certain MRI slices, which can occasionally baffle even standard convolutional architectures. Consequently, while the overall accuracy is high, the model's reliability in distinguishing between these specific aggressive tumors requires further refinement.

Another limitation involves the dataset and the resulting confidence levels in complex cases. Although clear-cut cases yielded confidence scores exceeding 0.98, more ambiguous scans resulted in significantly lower confidence, ranging from 0.54 to 0.58. This indicates that the model's certainty is highly dependent on the quality and typicality of the input image. Furthermore, the system is currently limited to a four-class categorization. In a real-world clinical setting, many other types of neurological anomalies and sub-types of tumors exist that the current model is not trained to recognize, potentially leading to "out-of-distribution" errors if an unsupported pathology is uploaded.

Finally, there are operational constraints regarding the system's current architecture. The backend processing, while efficient at approximately 2.3 seconds for a full result, relies on local file storage for heatmaps and a localized PostgreSQL instance. For a large-scale hospital implementation, the system would require transition to a distributed cloud infrastructure and more robust data encryption standards to meet international healthcare privacy regulations. Additionally, the Grad-CAM visualizations, while helpful, remain a "coarse" localization tool; they highlight general areas of importance but do not provide the pixel-perfect segmentation boundaries that a neurosurgeon might require for surgical planning.

7.4 Future Recommendations

To address the limitations identified during the development and testing of NeuroScan AI, several enhancements are proposed to move the system toward a more clinical-grade application. The first priority is to improve the model's discriminative power between Glioma and Meningioma cases. This can be achieved by implementing advanced architectures like Vision Transformers (ViTs) or ensemble models that combine the strengths of multiple neural networks. Incorporating a larger, multi-institutional dataset

with more diverse tumor sub-types would also help the model learn more nuanced features, reducing the confusion observed in the current performance report.

From a technical and feature-based perspective, the following enhancements are recommended:

- **Semantic Segmentation:** Future versions should transition from coarse Grad-CAM heatmaps to pixel-level segmentation masks. This would provide neurosurgeons with precise tumor boundaries and volume calculations, which are critical for surgical planning and monitoring tumor growth over time.
- **Longitudinal AI Analytics:** While the current system stores history, an advanced recommendation would be to implement "Comparison Analytics." This feature would allow the AI to analyze two different scans of the same patient from different dates and automatically report on whether the tumor has regressed, stabilized, or progressed.
- **Cloud Scalability and Security:** To support large-scale hospital deployments, the architecture should be migrated to a cloud-native environment using Docker and Kubernetes. Furthermore, implementing end-to-end encryption and FHIR (Fast Healthcare Interoperability Resources) standards would ensure the system complies with global health data privacy regulations like HIPAA or GDPR.
- **Mobile Clinician App:** Developing a mobile-optimized version of the frontend would allow clinicians to receive instant notifications and review AI-generated reports on tablet devices while on hospital rounds, increasing the system's accessibility.

By implementing these recommendations, NeuroScan AI can evolve from a diagnostic prototype into a comprehensive, high-precision clinical decision support system.

8 REFERENCES

- Abunadi, I., & Senan, N. (2024). Deep learning-based computer-aided diagnosis for brain tumor classification using MRI images. *Results in Engineering*, 21, 101830.
- Chollet, F. (2021). *Deep learning with Python* (2nd ed.). Manning Publications.
- Edureka!. (2021, June 27). *Cancer detection using deep learning | Deep learning projects | Deep learning training | Edureka* [Video]. YouTube.
<https://www.youtube.com/watch?v=7MceDfpnP8k>
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.
<http://www.deeplearningbook.org/>
- Google Developers. (n.d.). *Machine learning crash course*.
<https://developers.google.com/machine-learning/crash-course>
- Nayak, D. R., Padhy, N., Mallick, P. K., Zymbler, M., & Kumar, S. (2022). Brain tumor detection and classification using machine learning: A comprehensive survey. *Complex & Intelligent Systems*, 8(2), 1761–1788. <https://doi.org/10.1007/s40747-021-00563-y>
- Nickparvar, M. (2021). *Brain tumor MRI dataset* [Data set]. Kaggle.
<https://www.kaggle.com/datasets/masoudnickparvar/brain-tumor-mri-dataset>
- Rosebrock, A. (2017). *Deep learning for computer vision with Python*. PyImageSearch.
- Sarmah, A. M., & Laskar, S. (2021). Brain tumor detection using deep learning: A review. In *2021 International Conference on Advances in Computing, Control, and Telecommunication Technologies (ACT)* (pp. 150–155). IEEE.
<https://ieeexplore.ieee.org/document/9498384>

9 APPENDICES

A.1 GitHub Repository

All project files, including the CCN model and training and model weights, are available at:

<https://github.com/arsusan/Brain-tumor-detection-system>

A.2 Core CNN Architecture

```
class BrainTumorModel:

    def __init__(self, config: Config):

        self.config = config

        self.model = None

    def build_cnn(self, input_shape: tuple = None, dropout_rate: float = None) -> keras.Model:

        if input_shape is None:

            input_shape = self.config.INPUT_SHAPE

        if dropout_rate is None:

            dropout_rate = self.config.DROPOUT_RATE

        print(f"🐛 Building Functional 4-Class CNN with input shape: {input_shape}")

        inputs = layers.Input(shape=input_shape, name="mri_input")

        # Block 1

        x = layers.Conv2D(32, (3, 3), padding='same', name="conv2d_1")(inputs)

        x = layers.BatchNormalization()(x)

        x = layers.Activation('relu')(x)

        x = layers.Conv2D(32, (3, 3), padding='same', name="conv2d_2")(x)

        x = layers.BatchNormalization()(x)

        x = layers.Activation('relu')(x)

        x = layers.MaxPooling2D((2, 2))(x)

        x = layers.Dropout(dropout_rate * 0.5)(x)
```



```

# Block 2

x = layers.Conv2D(64, (3, 3), padding='same', name="conv2d_3")(x)

x = layers.BatchNormalization()(x)

x = layers.Activation('relu')(x)

x = layers.Conv2D(64, (3, 3), padding='same', name="conv2d_4")(x)

x = layers.BatchNormalization()(x)

x = layers.Activation('relu')(x)

x = layers.MaxPooling2D((2, 2))(x)

x = layers.Dropout(dropout_rate)(x)


# Block 3

x = layers.Conv2D(128, (3, 3), padding='same', name="conv2d_5")(x)

x = layers.BatchNormalization()(x)

x = layers.Activation('relu')(x)


# --- TARGET LAYER FOR GRAD-CAM ---

x = layers.Conv2D(128, (3, 3), padding='same', name="conv2d_6")(x)

x = layers.BatchNormalization()(x)

x = layers.Activation('relu')(x)

x = layers.MaxPooling2D((2, 2))(x)

x = layers.Dropout(dropout_rate)(x)


# Block 4

x = layers.Conv2D(256, (3, 3), padding='same', name="conv2d_final")(x)

x = layers.BatchNormalization()(x)

x = layers.Activation('relu')(x)

x = layers.GlobalAveragePooling2D()(x)

x = layers.Dropout(dropout_rate * 0.8)(x)

```

```

# Dense layers

x = layers.Dense(128, kernel_regularizer=keras.regularizers.l2(0.001))(x)

x = layers.BatchNormalization()(x)

x = layers.Activation('relu')(x)

x = layers.Dropout(dropout_rate * 0.6)(x)

```

APPENDIX B: DATASET DOCUMENTATION

B.1 Dataset Source

The primary data was sourced from the Brain Tumor MRI Dataset.

- Author: Masoud Nickparvar
- Platform: Kaggle
- Link: <https://www.kaggle.com/datasets/masoudnickparvar/brain-tumor-mri-dataset>

B.2 Data Distribution The dataset consists of 7,023 MRI images categorized into four classes: Glioma, Meningioma, Pituitary, and No Tumor.

APPENDIX C: SYSTEM SPECIFICATIONS

Table C.1: Computing Environment

Component	Specification
Processor	Intel Core i5/11 gen
GPU Accelerator	NVIDIA RTX
Python Version	3.10

Component	Specification
Libraries	TensorFlow 2.x, Keras, NumPy, OpenCV

APPENDIX D: MODEL PERFORMANCE LOGS

D.1 Training History

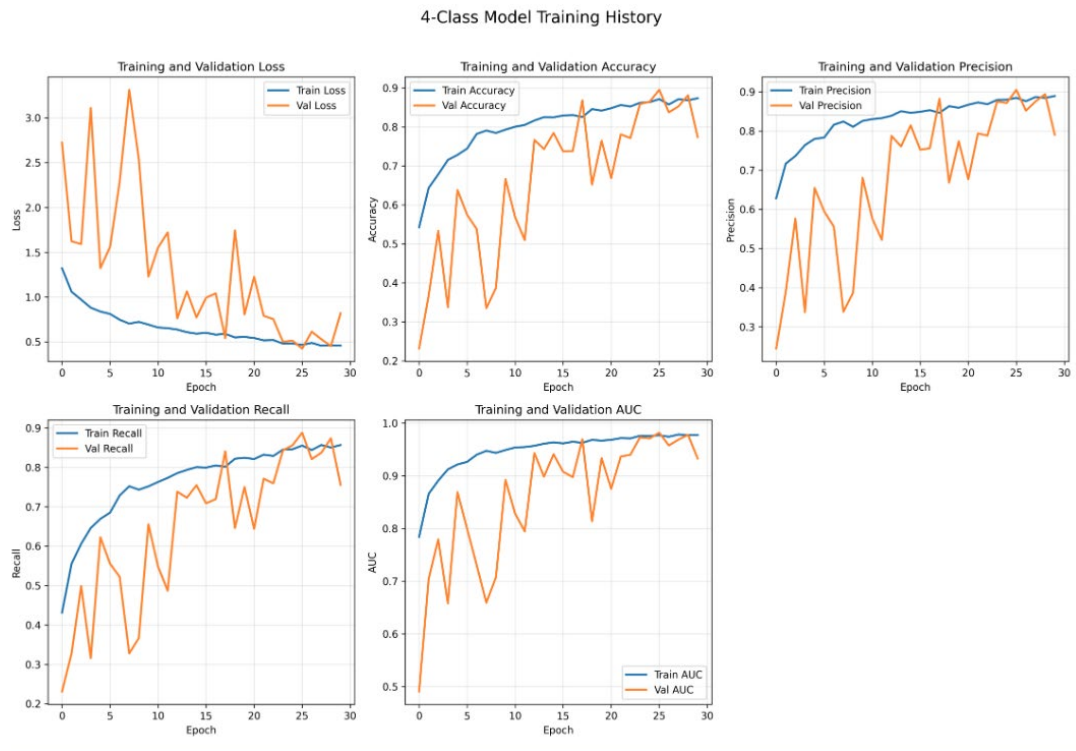


Figure D.1: Accuracy and Loss curves over 30 epochs

D.2 Final Metrics

- Overall Accuracy: 86%
- Loss Function: Categorical Cross-entropy